

Lecture Note 4: ESP32 – Multitasking and Deep Sleep

CSS452 Internet of Things

Dr. Seksan Laitrakun

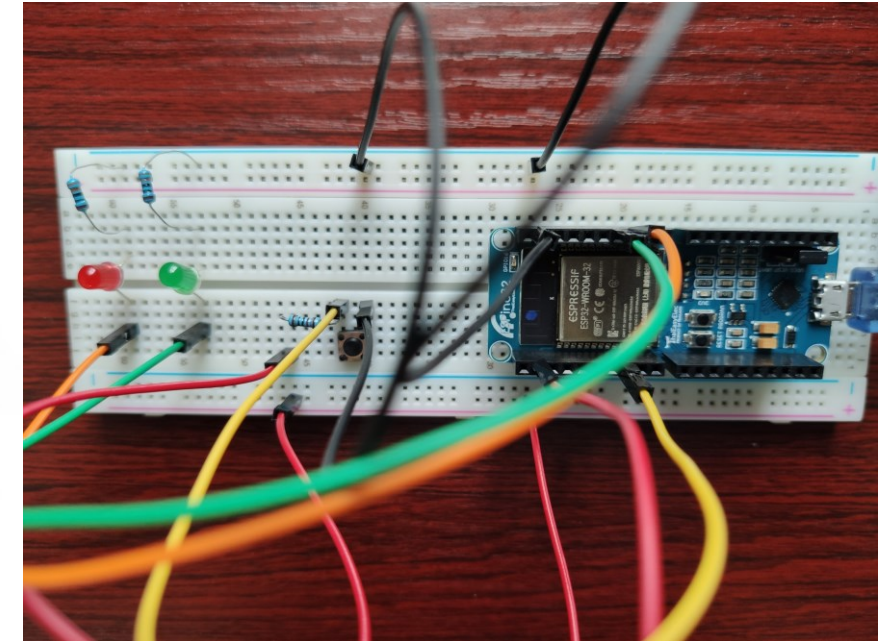
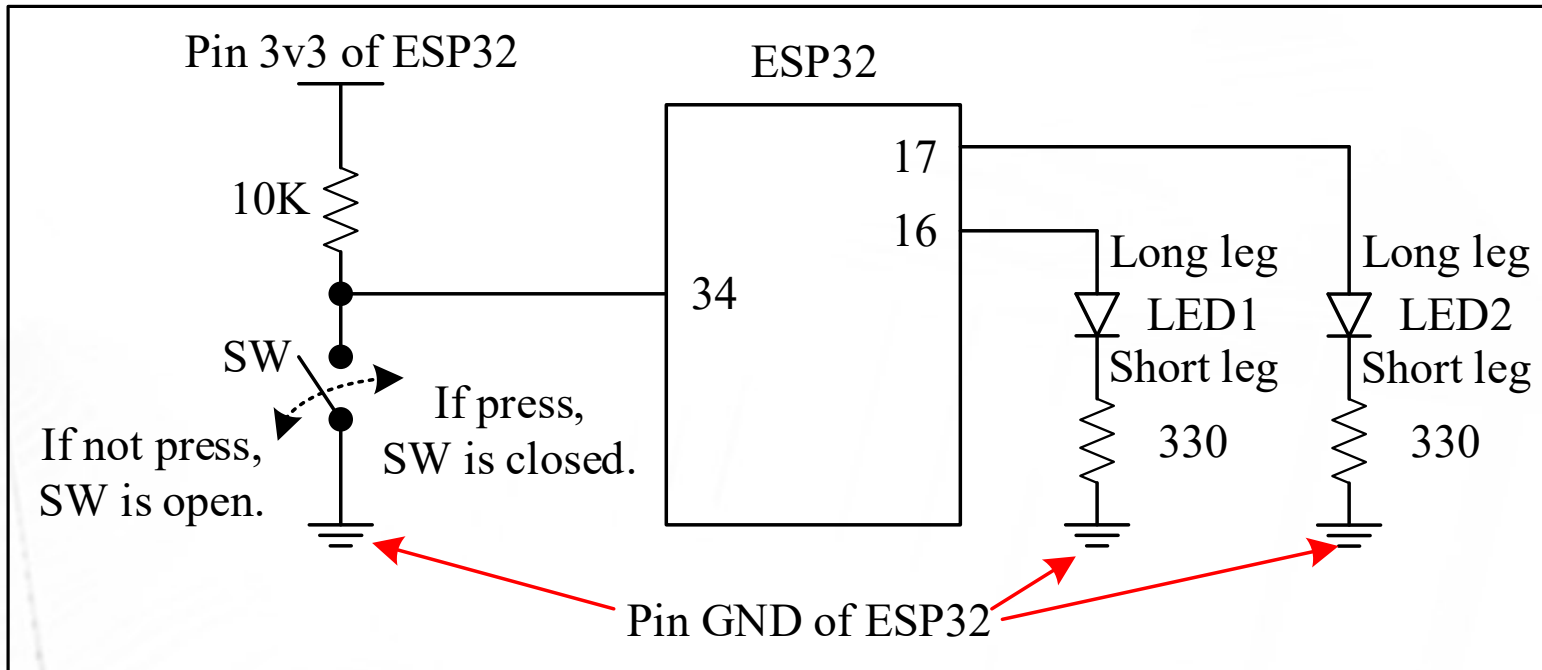
School of ICT, SIIT, Thammasat University

1. Multitasking: Timers and Interrupts
2. Multitasking: Task Management and Dual-Core Execution
3. Deep Sleep
4. Advanced Topics

- **Multitasking** in a microcontroller is the ability to run **multiple software routines or jobs** seemingly at the same time by rapidly switching the processor between them.
- ESP32 supports multitasking operations.
- In this lecture note, we will investigate on **writing programs for multitasking problems**.
- We study two task categories.
 - **Timing-Based Tasks**
 - Definition: **Actions that happen at regular, set time intervals**.
 - How it works: Uses a hardware timer, a clock, or a delay function to count time.
 - Example: Blinking an LED every 2 seconds.
 - **Event-Based Tasks**
 - Definition: **Actions that happen only when an external or internal event occurs**.
 - How it works: Uses interrupts or status checks to listen for a change.
 - Example: Turning on an LED when a user presses a button.

In Sections 1 & 2, use a variety of Arduino commands to solve this **multitasking problem**.

We connect a button and two LEDs to ESP32 as shown below. Note that, by connecting the switches in this way, pressing the switch gives the *LOW* logic at the ESP32 pin.



Then, we write an Arduino code to do the following 2 tasks in parallel: to control the LED1 and LED2.

- Task 1 (timing-based task): LED1 is alternatively turned on for 2 seconds and turned off for 2 seconds.
- Task 2 (event-based task): LED2 is turned on by pressing the switch; otherwise, it is turned off.

In Section 3: Deep Sleep

- **Deep sleep** is an ultra-low power mode in a microcontroller that turns off the main CPU core and most internal peripherals to save battery life.
- We will study a simple command to put an ESP32 to deep sleep.
- In addition, we will study four techniques (commands) to wake the ESP32 up from a deep sleep:
 - Timer Wake-up
 - Touch Wake-up
 - Ext0 External Wake-up
 - Ext1 External Wake-up

- Random Nerd Tutorials: <https://randomnerdtutorials.com/projects-esp32/>
- Last Minute Engineers: <https://lastminuteengineers.com/esp32-arduino-ide-tutorial/>
- Thai Easy Elect (Thai Language): <https://blog.thaieasyelec.com/espino32-article-table-of-contents/>
- ESP-IDF Programming Guide: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/index.html>

1. Multitasking: Timers and Interrupts

Multitasking is a technique to write a code such that several jobs will be executed concurrently (where the CPU jumps from one job to another job). This section consists of

1.1 Timers for Timing-Based Tasks

1.2 Interrupts for Event-Based Tasks

1.1 Timers for Timing-Based Tasks

1) Problem:

- In earlier examples, when we turn on or off an LED, we use the *delay()* function.
- However, the *delay()* function will block our code and not allow ESP32 to do anything else for the specified time duration.
- For example, *delay(1000)* means the ESP32 will stay at this command for 1 second.

2) Solution:

- On the other hand, to not waste the CPU time and if we have other work run concurrently, we could use a timer which is the *millis()* function.
- The *millis()* function. The *millis()* function will return the number of milliseconds that have passed since the program first started. For example,

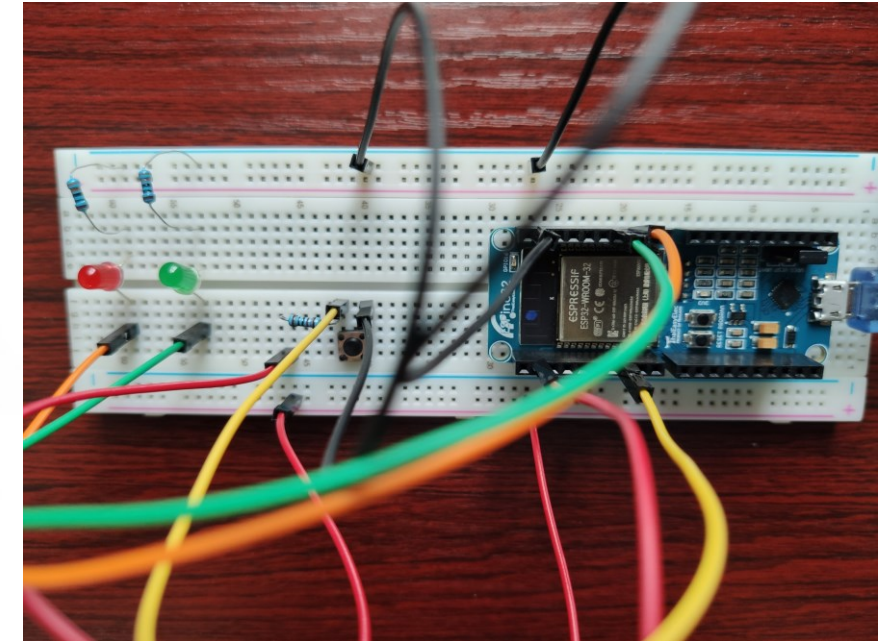
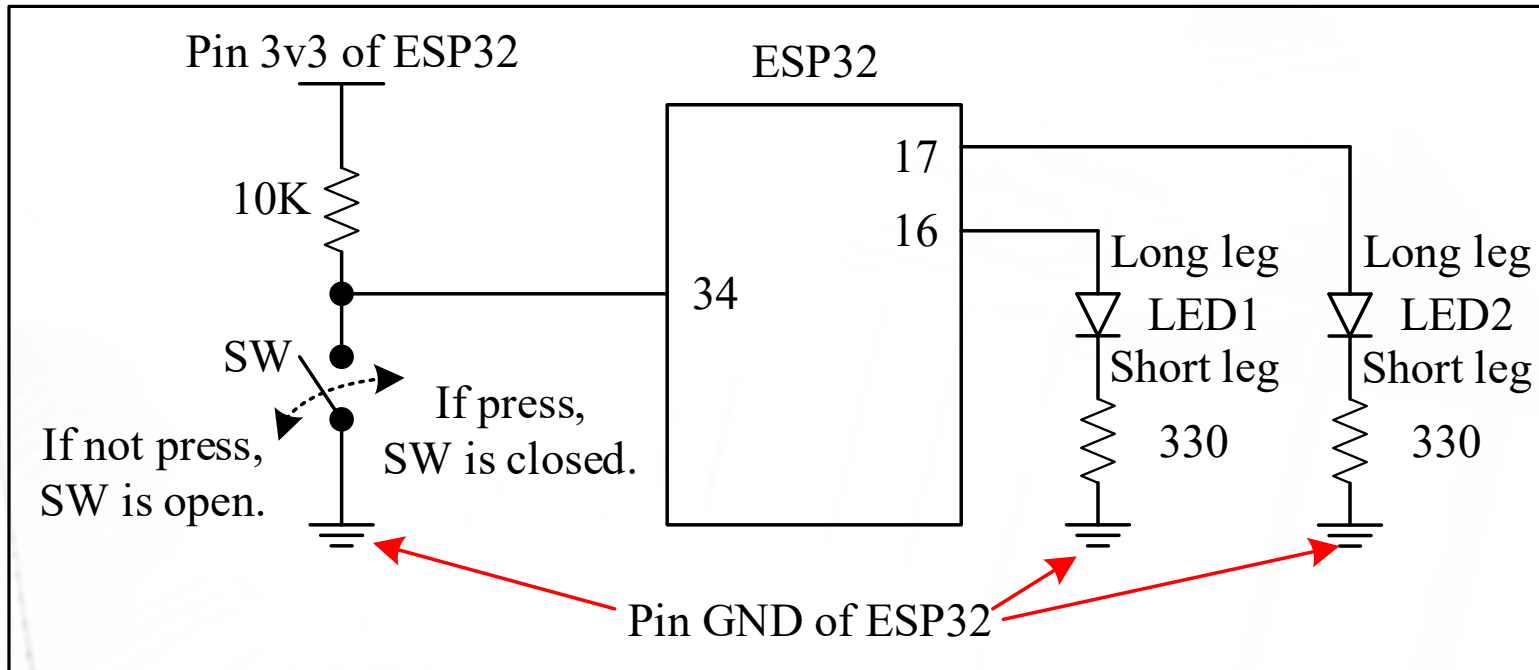
$X = \text{millis}();$

where X stores the number of milliseconds. An example is shown in the next slide.

1.1 Timers

Example 1

We connect a button and two LEDs to ESP32 as shown below. Note that, by connecting the switches in this way, pressing the switch gives the *LOW* logic at the ESP32 pin.



Then, we write an Arduino code to do the following 2 tasks in parallel: to control the LED1 and LED2.:

- Task 1 (timing-based task): LED1 is alternatively turned on for 2 seconds and turned off for 2 seconds.
- Task 2 (event-based task): LED2 is turned on by pressing the switch; otherwise, it is turned off.

1.1 Timers

Example 1 (Ex_millis.ino)

```
1 // set pin numbers
2 const int buttonPin = 34; // the push button pin
3 const int led1Pin = 16;    // the LED1 pin
4 const int led2Pin = 17;    // the LED2 pin
5
6 // variable for storing the pushbutton status
7 int buttonState = LOW;
8 int led1State = LOW;
9 unsigned long previousMillis = 0;
10 const long interval = 2000; // 2 second
11
12 void setup() {
13   pinMode(buttonPin, INPUT);
14   pinMode(led1Pin, OUTPUT);
15   pinMode(led2Pin, OUTPUT);
16 }
17
```

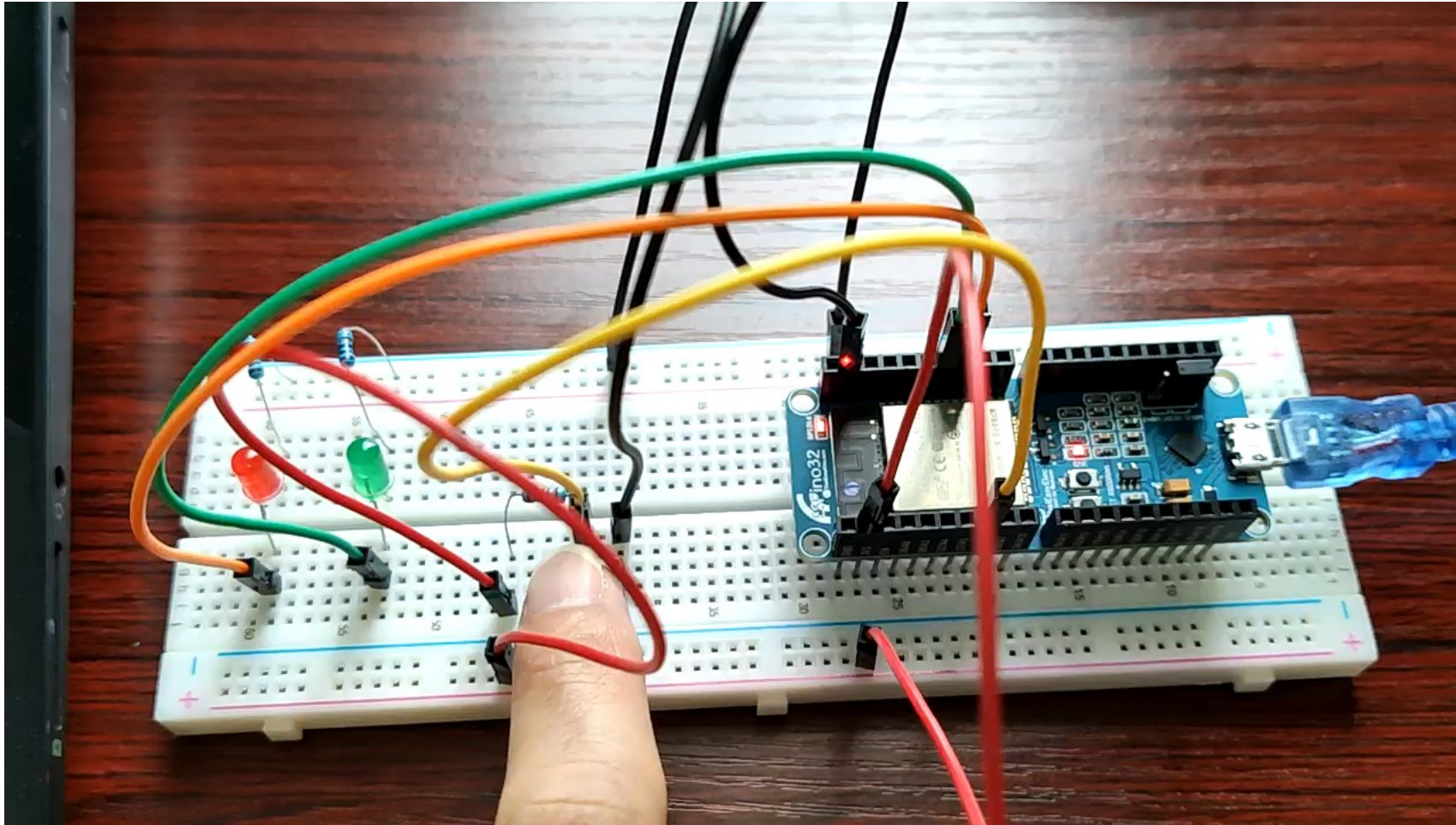
Use `milli()`; technique

```
18 void loop() {
19   // To turn on/off the LED1
20   unsigned long currentMillis = millis();
21   if (currentMillis - previousMillis >= interval) {
22     // save the last time you blinked the LED
23     previousMillis = currentMillis;
24     if (led1State == LOW) {
25       led1State = HIGH;
26     } else {
27       led1State = LOW;
28     }
29     digitalWrite(led1Pin, led1State);
30   }
31
32   // To turn on/off the LED2
33   buttonState = digitalRead(buttonPin);
34   if (buttonState == LOW) { // Press the switch
35     digitalWrite(led2Pin, HIGH); // LED2 on
36   } else {
37     digitalWrite(led2Pin, LOW); // LED2 off
38   }
39 }
```

1.1 Timers

Example 1

A demonstration video is shown below.



1.2 Interrupts for Event-Based Tasks

- **Interrupts** are useful for making things happen automatically in microcontroller programs, and can help solve event-based problems.
 - With interrupts we do not need to constantly check the current value of a pin.
- In this section, we will apply a **pin-change interrupt**.
 - **With pin-change interrupt, when a voltage change is detected at the pin, an event is triggered (a function is called).**
- Other types of interrupts are available such as timer interrupts (<https://www.arduino.cc/reference/en/libraries/timerone/>)

1.2 Interrupts

1. Pin-Change Interrupt

To set an interrupt in the Arduino IDE, we use the *attachInterrupt()* function (put it in the *setup()* function), whose syntax is

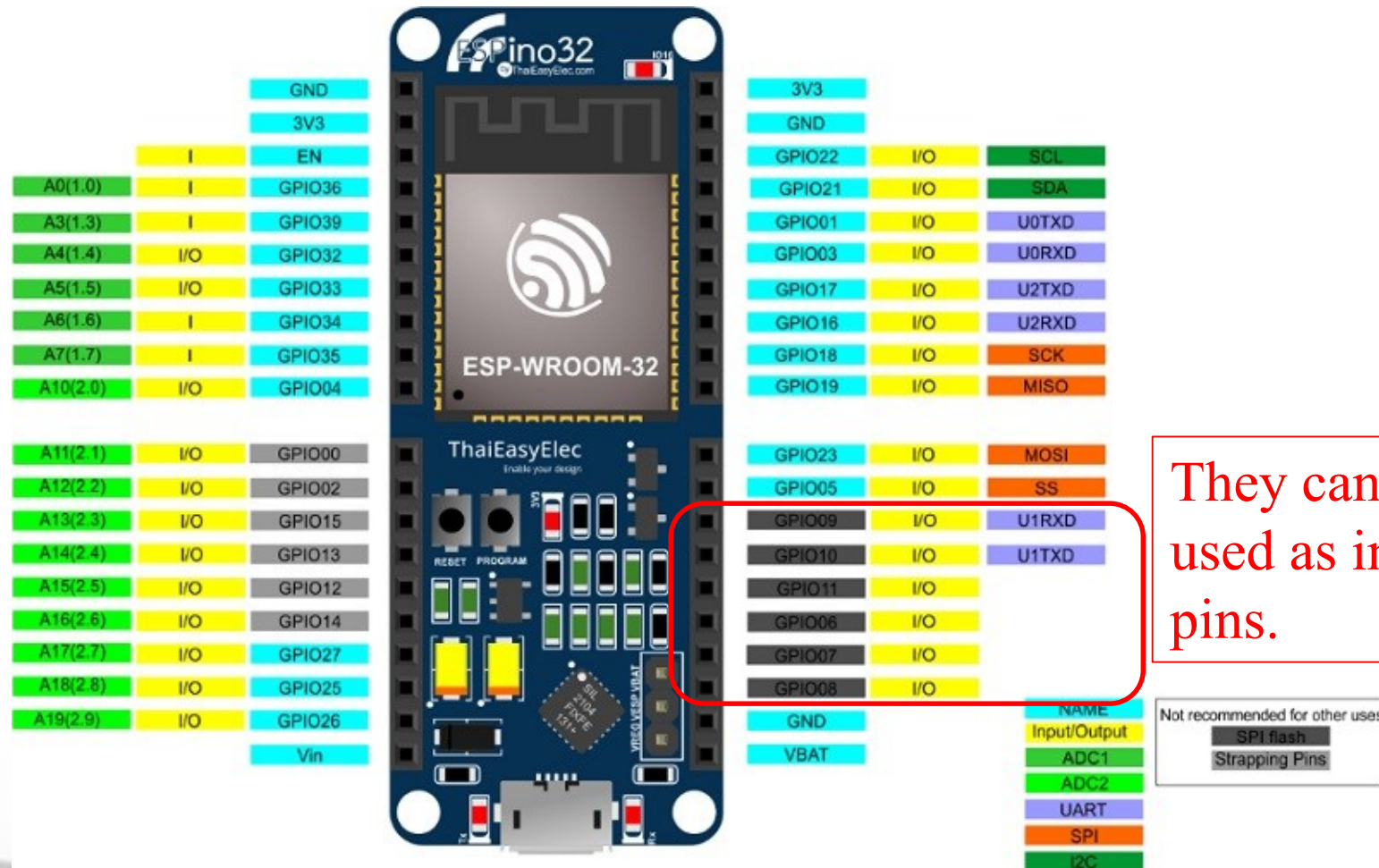
attachInterrupt(digitalPinToInterrupt(GPIO), function, mode);

Parameter	Meaning
<i>digitalPinToInterrupt(GPIO)</i>	A function is to set the actual <i>GPIO</i> as an interrupt pin. The <i>GPIO</i> pins useable as interrupt pins are show in the next page. This GPIO pin must be declared as an INPUT pin.
<i>function</i>	The second argument is the name of the function that will be called every time the interrupt is triggered.
<i>mode</i>	The third argument is the <i>mode</i> when the interrupt is triggered. There are 5 different modes: • LOW: to trigger the interrupt whenever the pin is LOW. • HIGH: to trigger the interrupt whenever the pin is HIGH. • CHANGE: to trigger the interrupt whenever the pin changes value – for example from HIGH to LOW or LOW to HIGH. • FALLING: for when the pin goes from HIGH to LOW. • RISING: to trigger when the pin goes from LOW to HIGH.

1.2 Interrupts

1. Pin-Change Interrupt

The GPIO pins useable as interrupt pins are all GPIO pins except the pins GPIO6 – GPIO11.

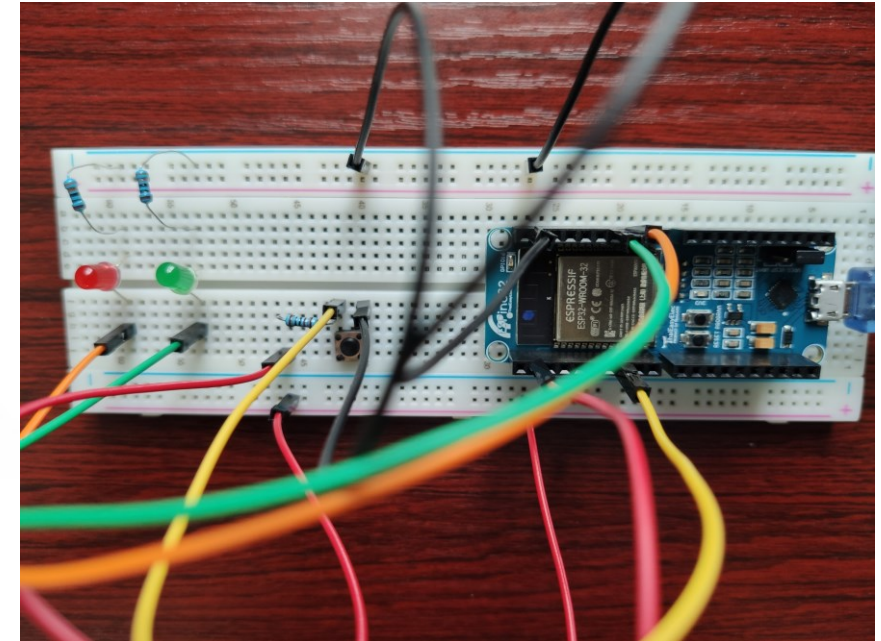
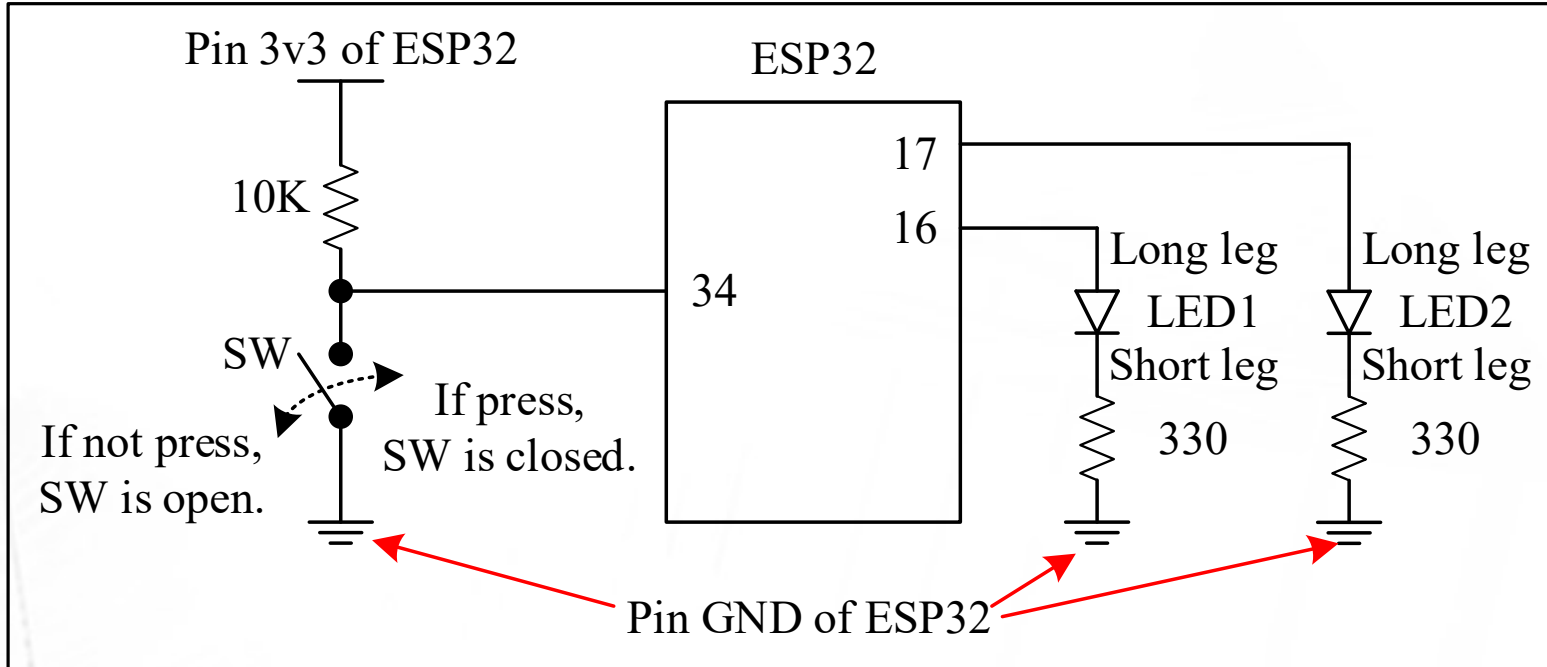


They cannot be used as interrupt pins.

1.2 Interrupts

Example 2

We connect a button and two LEDs to ESP32 as shown below (similar to Example 1).



Then, we write an Arduino code to do the following 2 tasks in parallel: to control the LED1 and LED2.

- Task 1 (timing-based task): LED1 is alternatively turned on for 2 seconds and turned off for 2 seconds.
- Task 2 (event-bases task): LED2 is **toggle**d when we press and release the switch (we will use an interrupt for this task). (Note that this task is differed from Example 1.)

1.2 Interrupts

Example 2 (Ex_Interrupts.ino)

By using an interrupt we can modify the code previously as follows.

```
1 // set pin numbers
2 const int buttonPin = 34; // the push button pin
3 const int led1Pin = 16;    // the LED1 pin
4 const int led2Pin = 17;    // the LED2 pin
5
6 // variable for storing the pushbutton status
7 int led1State = LOW;
8 int led2State = LOW;
9 unsigned long previousMillis = 0;
10 const long interval = 2000; // 2 second
11
12 void toggle() {
13     led2State = !led2State;
14     digitalWrite(led2Pin, led2State);
15 }
16
```

Step 1: Create a function.

1.2 Interrupts

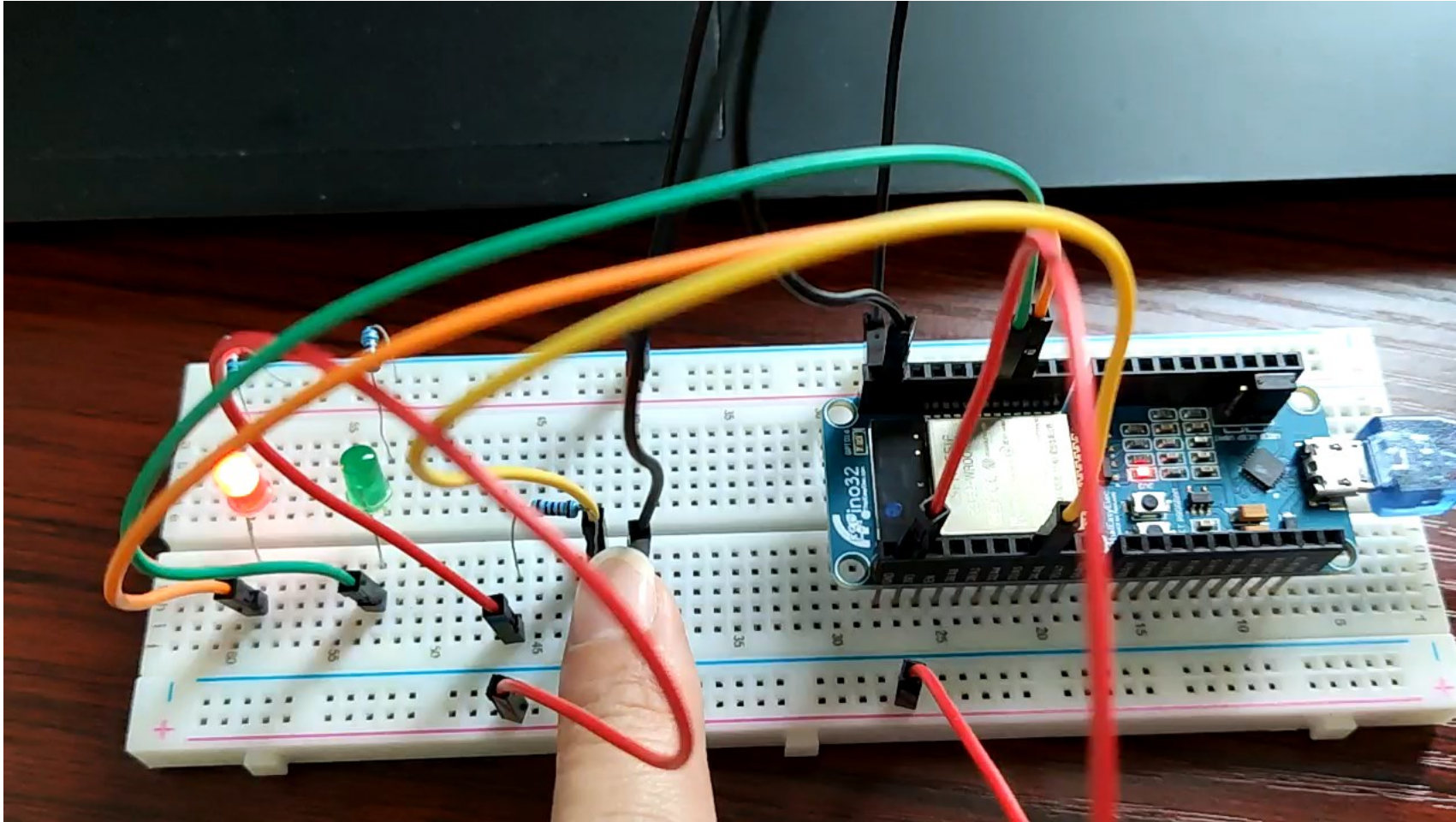
Example 2 (Ex_Interrupts.ino)

Step 2: Declare a pin-change interrupt.

```
17 void setup() {  
18   pinMode(buttonPin, INPUT);  
19   pinMode(led1Pin, OUTPUT);  
20   pinMode(led2Pin, OUTPUT);  
21   attachInterrupt(digitalPinToInterrupt(buttonPin), toggle, FALLING);  
22 }  
23  
24 void loop() {  
25   // To turn on/off the LED1  
26   unsigned long currentMillis = millis();  
27   if (currentMillis - previousMillis >= interval) {  
28     // save the last time you blinked the LED  
29     previousMillis = currentMillis;  
30     if (led1State == LOW) {  
31       led1State = HIGH;  
32     } else {  
33       led1State = LOW;  
34     }  
35     digitalWrite(led1Pin, led1State);  
36   }  
37 }
```

1.2 Interrupts Example 2

A demonstration video is shown below.



Topic	Detail	Command
1.1 Timers	Timing-based tasks	<code>millis();</code>
1.2 Interrupt	Event-based tasks	<code>attachInterrupt(digitalPinToInterrupt(GPIO), function, mode);</code>

2. Multitasking: Task Management and Dual-Core Execution

This section consists of

2.1 Task Management

2.2 ESP32 Dual Core

2.1 Task Management Commands

- **A problem:** As we have seen previously in Section 1, **we might assign ESP32 many tasks** and, then, **we have to organize them carefully such that these tasks can be executed concurrently.**
 - Specifically, in Example 1 (in Page 10), the programmer designs the order of task execution as follows: execute Task 1 first and then go to execute Task 2.
- **A solution:** In addition to managing and scheduling these jobs by ourselves, **we can define each job as a task (= a process) and let ESP32 manage all tasks by itself.**
 - To do this way, we have the following procedures.

2.1 Task Management Commands

- 1) We **create a task function for each task**. This function is similar to the functions we have declared before.

```
void TaskName (void *parameter) {  
    // Here, what ESP32 will do in this task.  
  
    ...  
    vTaskDelete(NULL);  
}
```

Note that:

- The *vTaskDelete(NULL)* function is needed to kill the task when finish.
- To run this task infinitely, we must create an infinite loop as shown below.

```
void TaskName (void *parameter) {  
    for (;;) {  
        // Here, what ESP32 will do in this job repeatedly.  
  
        ...  
    }  
    vTaskDelete(NULL);  
}
```

2.1 Task Management Commands

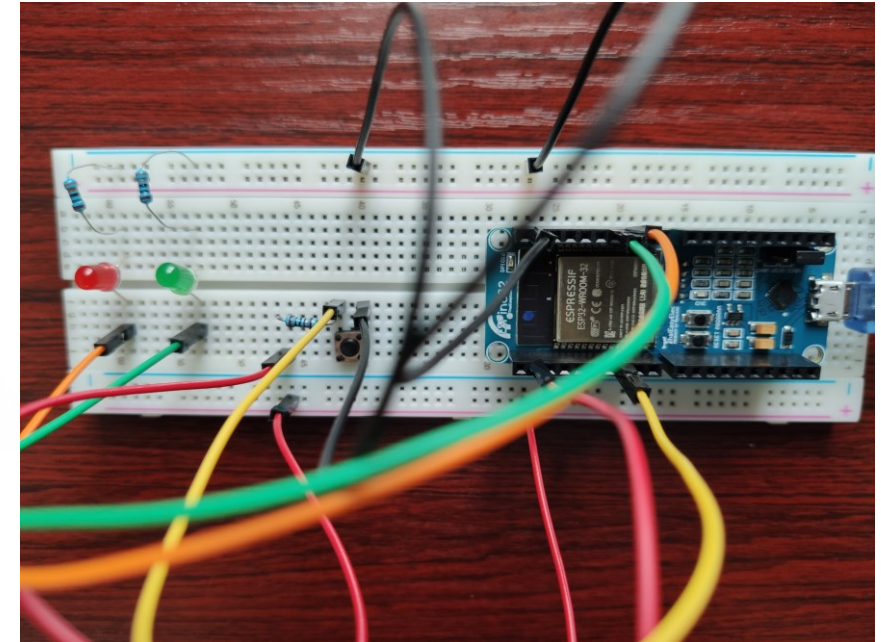
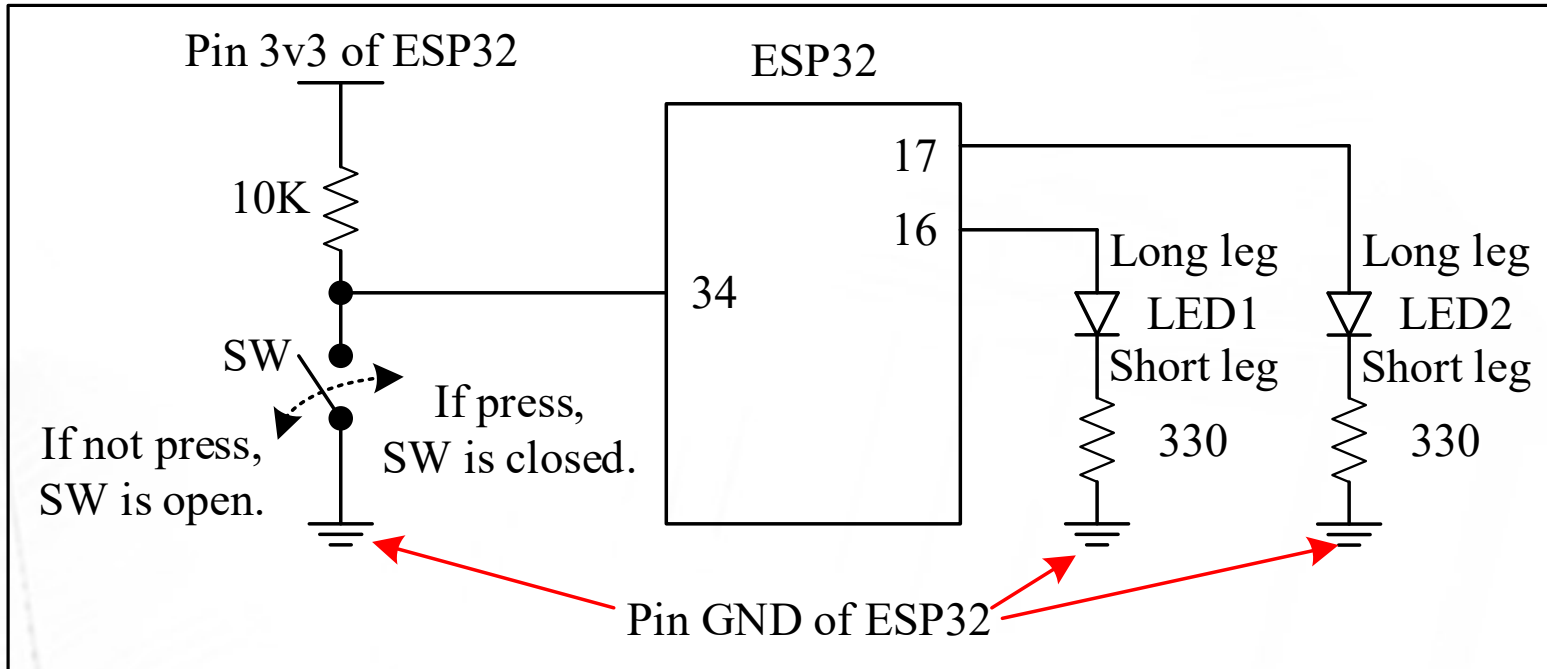
- 2) In the **setup() function**, we have to declare each task by using the ***xTaskCreate()*** function whose syntax is, for example,

```
xTaskCreate( TaskName,      /* Task function. */
             "TaskName",    /* Task name. */
             10000,          /* Stack size in bytes. */
             NULL,           /* Parameter passed as input of the task */
             1,              /* Priority of the task. */
             NULL            /* Task handle. */
             );
```

Parameter	Meaning
<i>Task function</i>	It is the name of the function for this job that we will ask ESP32 to do
<i>Task name</i>	It is the string of the function name
<i>Stack size in bytes</i>	It is the amount of memory that is allocated for this task
<i>Parameter</i>	a list of inputs which will be passed to the function above
<i>Priority of the task</i>	It is a number between 0 – 24 (higher number → higher priority which will be executed first when ESP32 needs to choose)
<i>Task handle</i>	This parameter stores an handle that can be used for latter reference of the task on calls to functions (for example, to delete a task or change its priority).

2.1 Task Management Example 3

We connect a button and two LEDs to ESP32 as shown below (similar to Example 1).



Then, we write an Arduino code to control the LED1 and LED2 by using tasks:

- Task 1: LED1 is alternatively turned on for 2 seconds and turned off for 2 seconds.
- Task 2: LED2 is on or off by pressing the switch (pressing the switch → LED2 on; otherwise, LED2 off).

2.1 Task Management

Example 3 (Ex_Multitasking.ino)

```
1 // set pin numbers
2 const int buttonPin = 34; // the push button pin
3 const int led1Pin = 16;    // the LED1 pin
4 const int led2Pin = 17;    // the LED2 pin
5
6 // variable for storing the pushbutton status
7 int buttonState = LOW;
8 int led1State = LOW;
9 int led2State = LOW;
10 unsigned long previousMillis = 0;
11 const long interval = 2000; // 2 second
12
```

Step 1: Create a function for Task 1.

```
13 void TimerLED(void * parameter){
14     for (;;) {
15         unsigned long currentMillis = millis();
16         if (currentMillis - previousMillis >= interval) {
17             // save the last time you blinked the LED
18             previousMillis = currentMillis;
19             if (led1State == LOW) {
20                 led1State = HIGH;
21             } else {
22                 led1State = LOW;
23             }
24         }
25         digitalWrite(led1Pin, led1State);
26     }
27     vTaskDelete(NULL);
28 }
29
```


2.1 Task Management

Example 3 (Ex_Multitasking.ino)

```
30 void SwitchLED(void * parameter) {
31     for (;;) {
32         buttonState = digitalRead(buttonPin);
33         if (buttonState == LOW) { // Press the switch
34             digitalWrite(led2Pin, HIGH); // LED2 on
35         } else {
36             digitalWrite(led2Pin, LOW); // LED2 off
37         }
38     }
39     vTaskDelete(NULL);
40 }

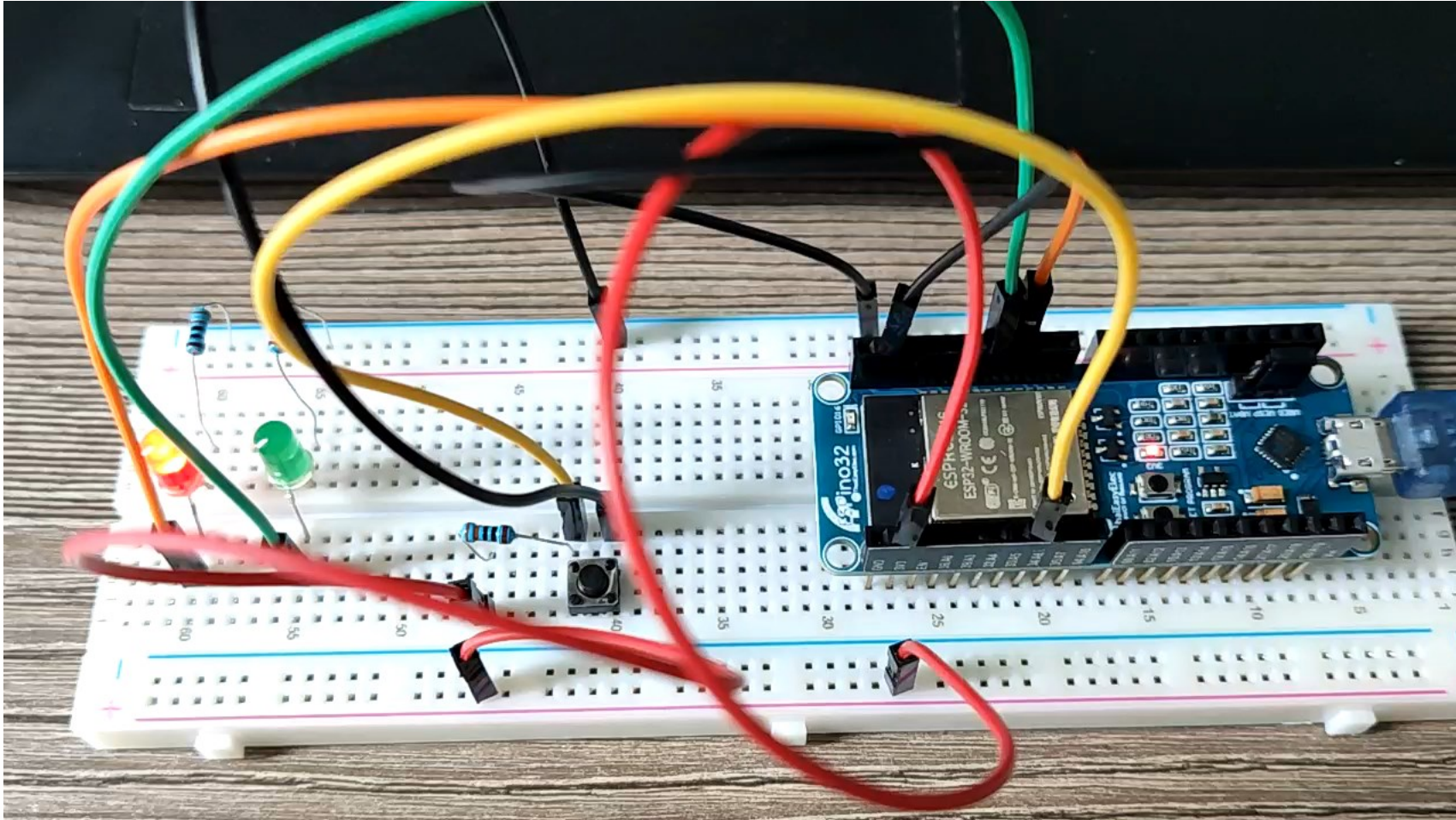
41
42 void setup() {
43     pinMode(buttonPin, INPUT);
44     pinMode(led1Pin, OUTPUT);
45     pinMode(led2Pin, OUTPUT);
46     xTaskCreate(TimerLED, "TimerLED", 1024, NULL, 1, NULL);
47     xTaskCreate(SwitchLED, "SwitchLED", 1024, NULL, 1, NULL);
48 }
49
50 void loop() {
51 }
```

Step 1: Create a function for Task 2.

Step 2: Declare each task by using the xTaskCreate().

2.1 Task Management Example 3

A demonstration video is shown below.



2.2 ESP32 Dual Core

1. Show the Running Core

- The ESP32 comes with 2 Xtensa 32-bit LX6 microprocessors: core 0 and core 1. So, it is dual core.
- When we run an Arduino code, we can monitor which core is running the current command by using

xPortGetCoreID();

2.2 ESP32 Dual Core

Example 4 (Ex_ShowCore.ino)

We connect the same circuit as shown in Example 3 and modify its Arduino program in Example 3 such that we show the running core on the serial monitor.

```
1 // set pin numbers
2 const int buttonPin = 34; // the push button pin
3 const int led1Pin = 16;   // the LED1 pin
4 const int led2Pin = 17;   // the LED2 pin
5
6 // variable for storing the pushbutton status
7 int buttonState = LOW;
8 int led1State = LOW;
9 int led2State = LOW;
10 unsigned long previousMillis = 0;
11 const long interval = 2000; // 2 second
12
```

2.2 ESP32 Dual Core Example 4 (Ex_ShowCore.ino)

```
13 void TimerLED(void * parameter){  
14     for (;;) {  
15         Serial.print("TimerLED() running on core ");  
16         Serial.println(xPortGetCoreID());  
17         delay(100);  
18         unsigned long currentMillis = millis();  
19         if (currentMillis - previousMillis >= interval) {  
20             // save the last time you blinked the LED  
21             previousMillis = currentMillis;  
22             if (led1State == LOW) {  
23                 led1State = HIGH;  
24             } else {  
25                 led1State = LOW;  
26             }  
27         }  
28         digitalWrite(led1Pin, led1State);  
29     }  
30     vTaskDelete(NULL);  
31 }  
32
```

Print the CPU core number.

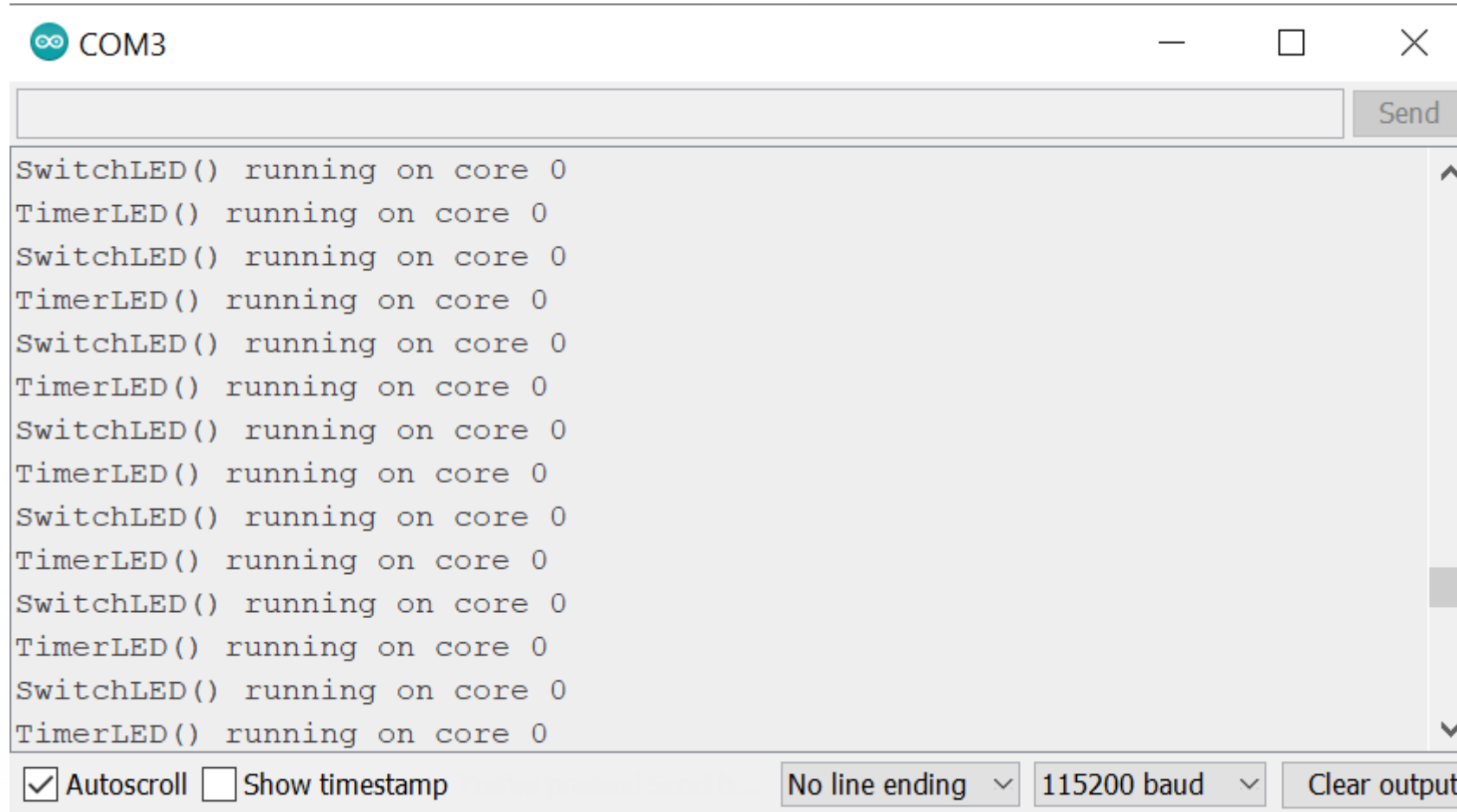
2.2 ESP32 Dual Core

```
33 void SwitchLED(void * parameter) {
34     for (;;) {
35         Serial.print("SwitchLED() running on core ");
36         Serial.println(xPortGetCoreID());
37         delay(100);
38         buttonState = digitalRead(buttonPin);
39         if (buttonState == LOW) { // Press the switch
40             digitalWrite(led2Pin, HIGH); // LED2 on
41         } else {
42             digitalWrite(led2Pin, LOW); // LED2 off
43         }
44     }
45     vTaskDelete(NULL);
46 }
47
48 void setup() {
49     Serial.begin(115200);
50     pinMode(buttonPin, INPUT);
51     pinMode(led1Pin, OUTPUT);
52     pinMode(led2Pin, OUTPUT);
53     xTaskCreate(TimerLED, "TimerLED", 1024, NULL, 1, NULL);
54     xTaskCreate(SwitchLED, "SwitchLED", 1024, NULL, 1, NULL);
55 }
56
57 void loop() {
58 }
```

Print the CPU core number.

2.2 ESP32 Dual Core Example 4

The result is shown on the serial monitor.



```
COM3  
SwitchLED() running on core 0  
TimerLED() running on core 0  
SwitchLED() running on core 0  
TimerLED() running on core 0  
SwitchLED() running on core 0  
TimerLED() running on core 0  
SwitchLED() running on core 0  
TimerLED() running on core 0  
SwitchLED() running on core 0  
TimerLED() running on core 0  
SwitchLED() running on core 0  
TimerLED() running on core 0  
SwitchLED() running on core 0  
TimerLED() running on core 0
```

☒ Autoscroll ☐ Show timestamp No line ending 115200 baud Clear output

2.2 ESP32 Dual Core

2. Assigning a Core to a Task

As shown in 2.1 where we can assign each job to a task function. Further, we can assign a core to each task function that we created. The steps are as follows (similar to what we have done in 2.1).

- 1) We declare a task function for each job (the same way we have done in 2.1, Page 18).
- 2) In the *setup()* function, we have to declare each task by using the function below whose syntax is, for example,

```
xTaskCreatePinnedToCore( TaskName,      /* Task function. */  
                        "TaskName", /* Task name. */  
                        10000,      /* Stack size in bytes. */  
                        NULL,      /* Parameter passed as input of the task */  
                        1,          /* Priority of the task. */  
                        NULL,      /* Task handle. */  
                        0           /* Core 0 */  
);
```

The input parameters are similar to those in the *xTaskCreate()* function (on Page 19) except the last one which we use to assign the core for this task. The number here can be either 0 or 1 (= Core 0 or 1).

2.2 ESP32 Dual Core

Example 5 (Ex_AssignCore.ino)

We connect the same circuit as shown in Example 3 and modify its Arduino program in Example 4 such that **we assign Core 0 to the *TimerLED* task and Core 1 to the *SwitchLED* task**. We also show the running core on the serial monitor. Note that, unlike the previous example where both tasks are executed by the Core 0 (concurrently), now each task is executed by different core. Both tasks are running parallelly/simultaneously.

```
1 // set pin numbers
2 const int buttonPin = 34;  // the push button pin
3 const int led1Pin = 16;    // the LED1 pin
4 const int led2Pin = 17;    // the LED2 pin
5
6 // variable for storing the pushbutton status
7 int buttonState = LOW;
8 int led1State = LOW;
9 int led2State = LOW;
10 unsigned long previousMillis = 0;
11 const long interval = 2000; // 2 second
12
```

2.2 ESP32 Dual Core Example 5 (Ex_AssignCore.ino)

```
13 void TimerLED(void * parameter){
14     for (;;) {
15         Serial.print("TimerLED() running on core ");
16         Serial.println(xPortGetCoreID());
17         delay(100);
18         unsigned long currentMillis = millis();
19         if (currentMillis - previousMillis >= interval) {
20             // save the last time you blinked the LED
21             previousMillis = currentMillis;
22             if (led1State == LOW) {
23                 led1State = HIGH;
24             } else {
25                 led1State = LOW;
26             }
27         }
28         digitalWrite(led1Pin, led1State);
29     }
30     vTaskDelete(NULL);
31 }
32
```

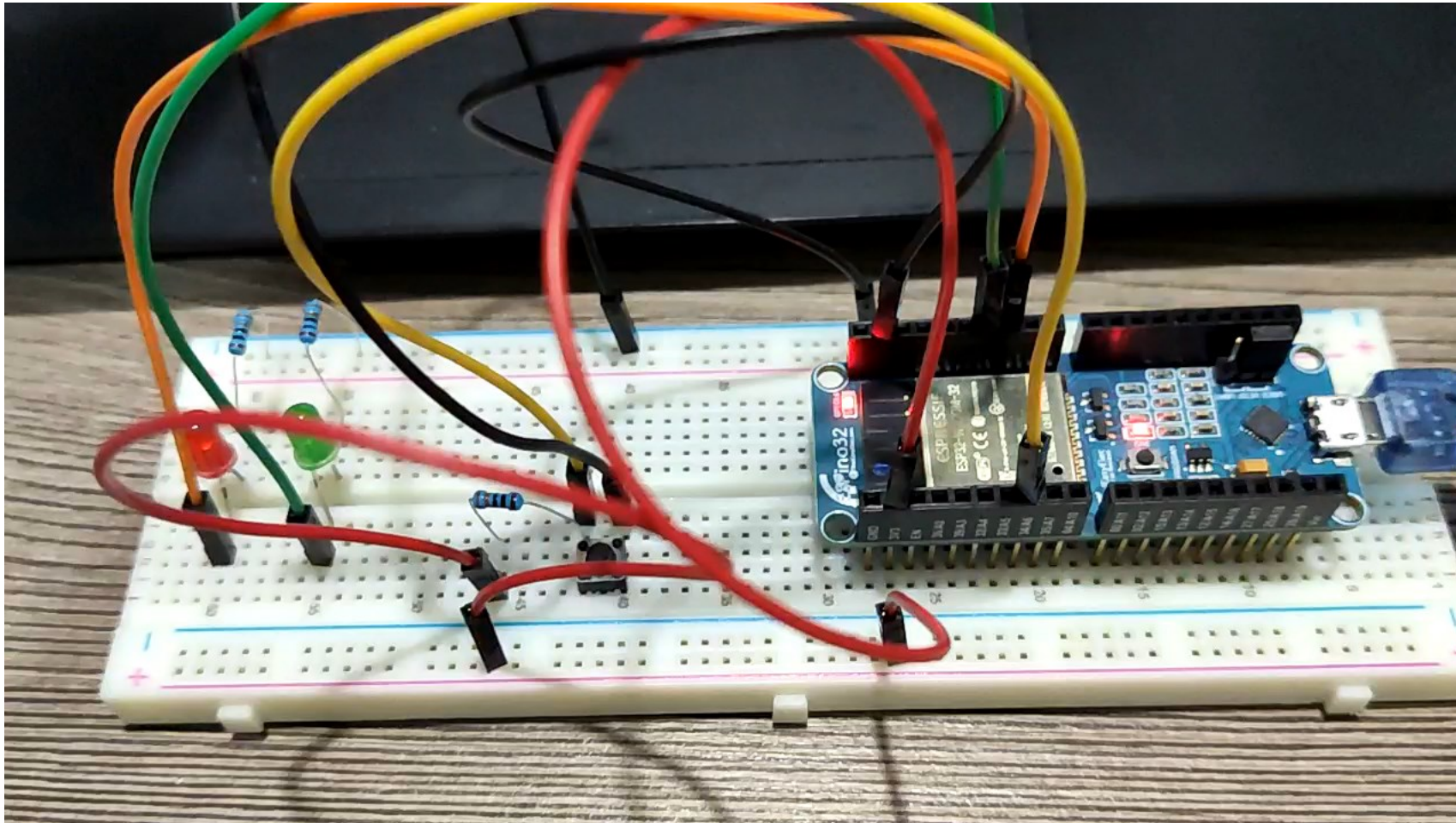

2.2 ESP32 Dual Core

```
33 void SwitchLED(void * parameter) {
34     for (;;) {
35         Serial.print("SwitchLED() running on core ");
36         Serial.println(xPortGetCoreID());
37         delay(100);
38         buttonState = digitalRead(buttonPin);
39         if (buttonState == LOW) { // Press the switch
40             digitalWrite(led2Pin, HIGH); // LED2 on
41         } else {
42             digitalWrite(led2Pin, LOW); // LED2 off
43         }
44     }
45     vTaskDelete(NULL);
46 }
47
48 void setup() {
49     Serial.begin(115200);
50     pinMode(buttonPin, INPUT);
51     pinMode(led1Pin, OUTPUT);
52     pinMode(led2Pin, OUTPUT);
53     xTaskCreatePinnedToCore(TimerLED, "TimerLED", 1024, NULL, 1, NULL, 0);
54     xTaskCreatePinnedToCore(SwitchLED, "SwitchLED", 1024, NULL, 1, NULL, 1);
55 }
56
57 void loop() {
58 }
```

Declare the tasks and assign the cores.

2.2 ESP32 Dual Core Example 5

A demonstration video is shown below.



Topic	Detail	Command
2.1 Task Management	Create a task as a function	-
2. ESP32 Dual Core	Show the current running core	xPortGetCoreID();
	Assigning a core to a task	xTaskCreatePinnedToCore(...);

3. Deep Sleep

This section consists of

3.1 ESP32 Power Modes

3.2 Deep Sleep

3.3 Timer Wake-up

3.4 Touch Wake-up

3.5 Ext0 External Wake-up

3.6 Ext1 External Wake-up

3.1 ESP32 Power Modes

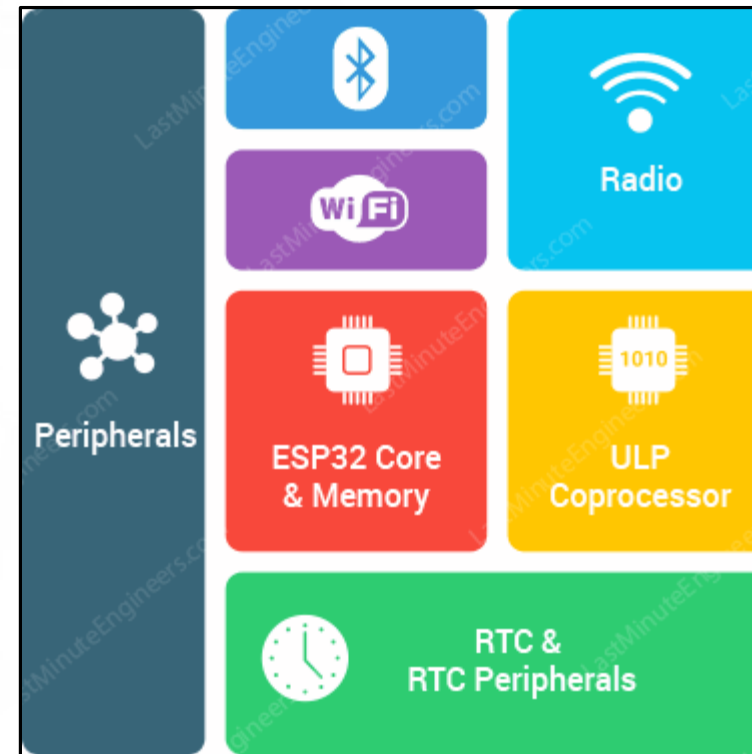
ESP32 is a popular microcontroller used in many projects. However, ESP32 can be a relatively power-hungry device. It usually pulls about 75mA in normal operation and hits about 240mA while transmitting data over WiFi.

- When our IoT project is powered by a plug in the wall, we tend not to care too much about power consumption. But if we are going to power your project by batteries, every mA counts.
- The solution here is to cut back ESP32's power usage by leveraging one of its Sleep Modes. It's really a great strategy for dramatically extending the battery life of a project that does not need to be active all the time.

3.1 ESP32 Power Modes

1. Inside the Chip

In order to understand how ESP32 achieves power saving, we need to know what is inside the chip. The following illustration shows function block diagram of ESP32 chip. At the heart of the ESP32 chip is a Dual-Core 32-bit microprocessor along with 448 KB of ROM, 520 KB of SRAM and 4MB of Flash memory. It also contains WiFi module, Bluetooth Module, the ULP (Ultra Low Power) coprocessor, the RTC ((Real-Time Clock)) module, and lot of peripherals

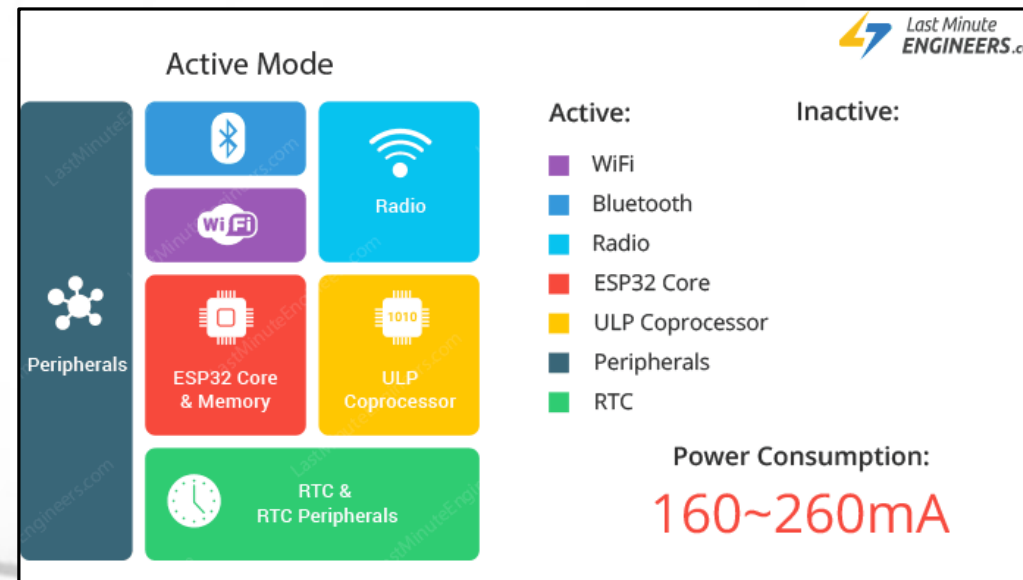


3.1 ESP32 Power Modes

2. Power Modes

ESP32 offers 5 configurable power modes. As per the power requirement, the chip can switch between different power modes. The modes are: Active Mode, Modem Sleep Mode, Light Sleep Mode, Deep Sleep Mode, Hibernation Mode.

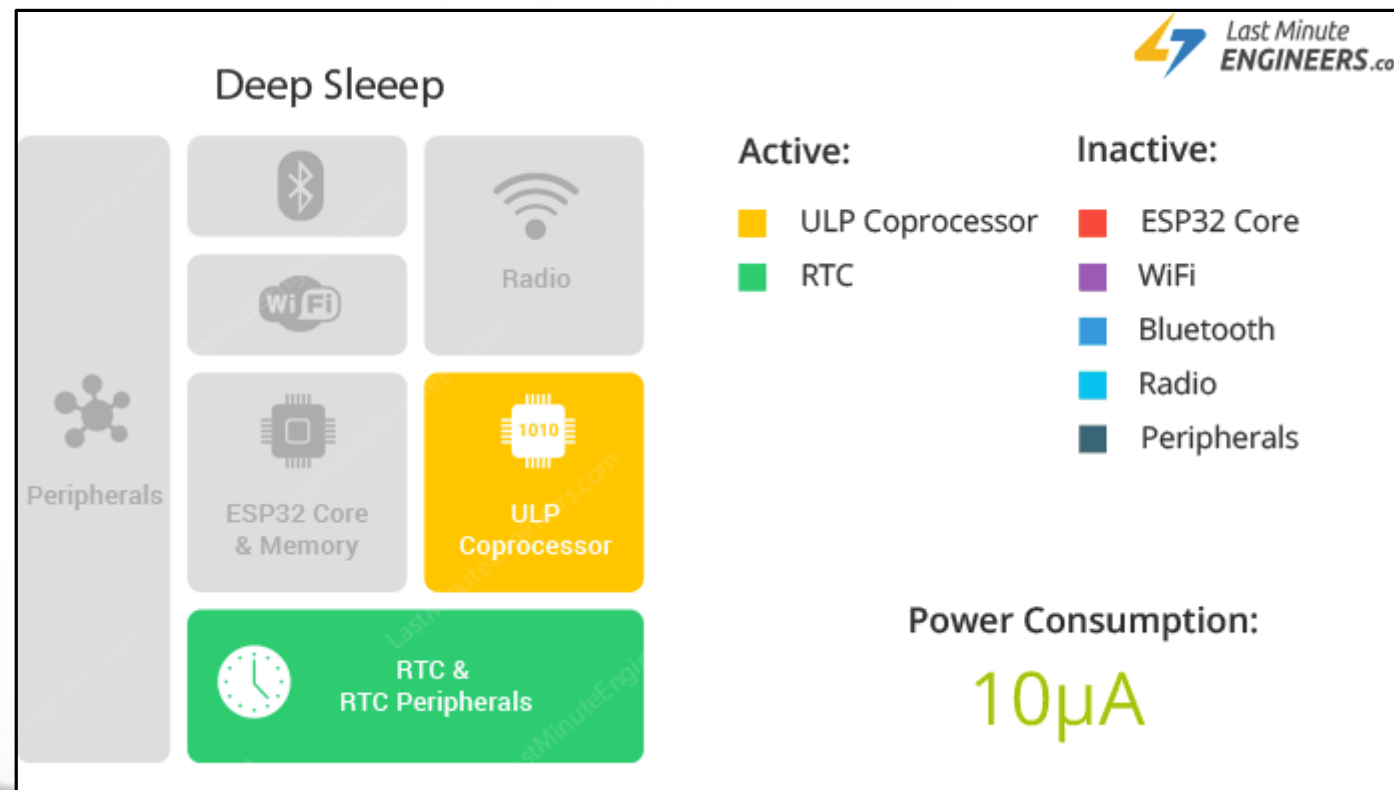
- The normal mode is also known as Active Mode. In this mode all the features of the chip are active. As the active mode keeps everything (especially the WiFi module, the Processing Cores and the Bluetooth module) ON at all times, the chip requires more than 240mA current to operate.



3.1 ESP32 Power Modes

2. Power Modes

- In deep sleep mode, the CPU, most of the RAM and all the digital peripherals are powered off. **The only parts of the chip that remains powered on are: RTC controller, RTC peripherals (including ULP co-processor), and RTC memories (slow and fast).** The chip consumes around 0.15 mA (if ULP co-processor is powered on) to 10 μ A.



3.2 Deep Sleep

1. Explanation

During deep sleep mode:

- The main CPU is powered down.
- Along with the CPU, the main memory of the chip is also disabled. So, everything stored in that memory is wiped out and cannot be accessed.
- However, the RTC memory is kept powered on. So, its contents are preserved during deep sleep and can be retrieved after we wake the chip up.
- So, if we want to use the data over reboot, store it into the RTC memory by defining a global variable with RTC_DATA_ATTR attribute. For example,
$$RTC_DATA_ATTR \text{ int } bootCount = 0;$$
- In Deep sleep mode, power is shut off to the entire chip except RTC module. So, any data that is not in the RTC recovery memory is lost.
- When the chip is waken up, it will restart with a reset. This means program execution starts from the beginning once again.

3.2 Deep Sleep

1. Explanation

- We use this **function to let ESP32 go to the deep sleep mode:**

esp_deep_sleep_start();

- Note that whenever ESP32 executes this function, it will immediately go to the deep sleep mode and does not execute any codes that are after it.
- When the chip is waken up, it will restart with a reset.
- This function can be put inside either the *setup()* or *loop()*.

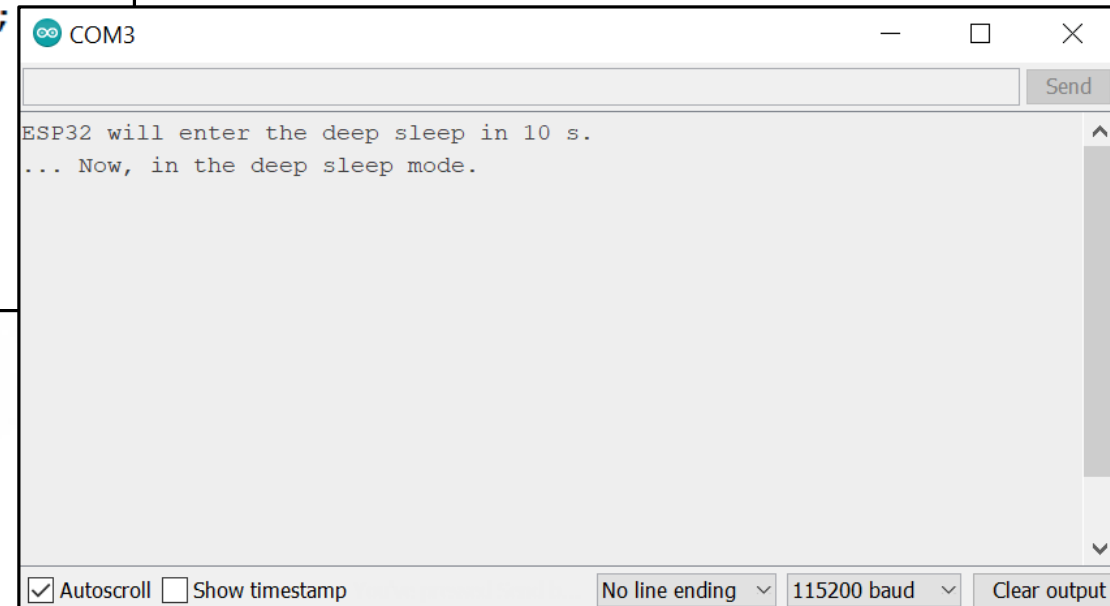
3.2 Deep Sleep

Example 6 (Ex_DeepSleep.ino)

Write this code and upload to ESP32. Open the serial monitor to see the result. No circuit connection is needed.

```
1 void setup() {  
2   Serial.begin(115200);  
3   delay(1000); //Take some time to open up the Serial Monitor  
4   Serial.println("ESP32 will enter the deep sleep in 10 s.");  
5   delay(10000);  
6   //Go to sleep now  
7   Serial.println "... Now, in the deep sleep mode.");  
8   esp_deep_sleep_start();  
9   Serial.println("This sentence will never be printed.");  
10 }  
11  
12 void loop() {  
13   //ESP32 will never come here.  
14 }
```

Using the deep sleep.



In the previous example, the chip will be in deep sleep mode indefinitely, until external reset is applied. However, **wake up from deep sleep mode can be done using several sources/methods**. Here, we will study the following four techniques:

- 1) Timer
- 2) Touch Pin
- 3) Ext0 External Wake Up – Use it when you want to wake-up the chip by one particular pin only.
- 4) Ext1 External Wake Up – Use it when you will use several pins for the wake-up.

3.2 Deep Sleep

2. ESP32 Deep Sleep Wake-up Sources

Remark:

- 1) Wake-up sources can be combined, in this case the chip will wake up when any one of the sources is triggered.
- 2) These sources should be configured at any moment before entering in to the deep sleep mode (i.e., before executing the *esp_deep_sleep_start()* function).

3.3 Timer Wake-Up

1. The Command

RTC controller has a built in timer which can be used to wake up the chip after a predefined amount of time. Time is specified at microsecond precision, but the actual resolution depends on the clock source selected. We use the following function:

*esp_sleep_enable_timer_wakeup($X * 1000000$) ;*

where X is the amount of time in the unit of seconds.

3.3 Timer Wake-Up

Example 7 (Ex_TimerWakeUp.ino)

We will write an Arduino program such that ESP32 will alternatively wake up for 10 seconds and deep sleep for 5 seconds by using the timer wake up.

Before

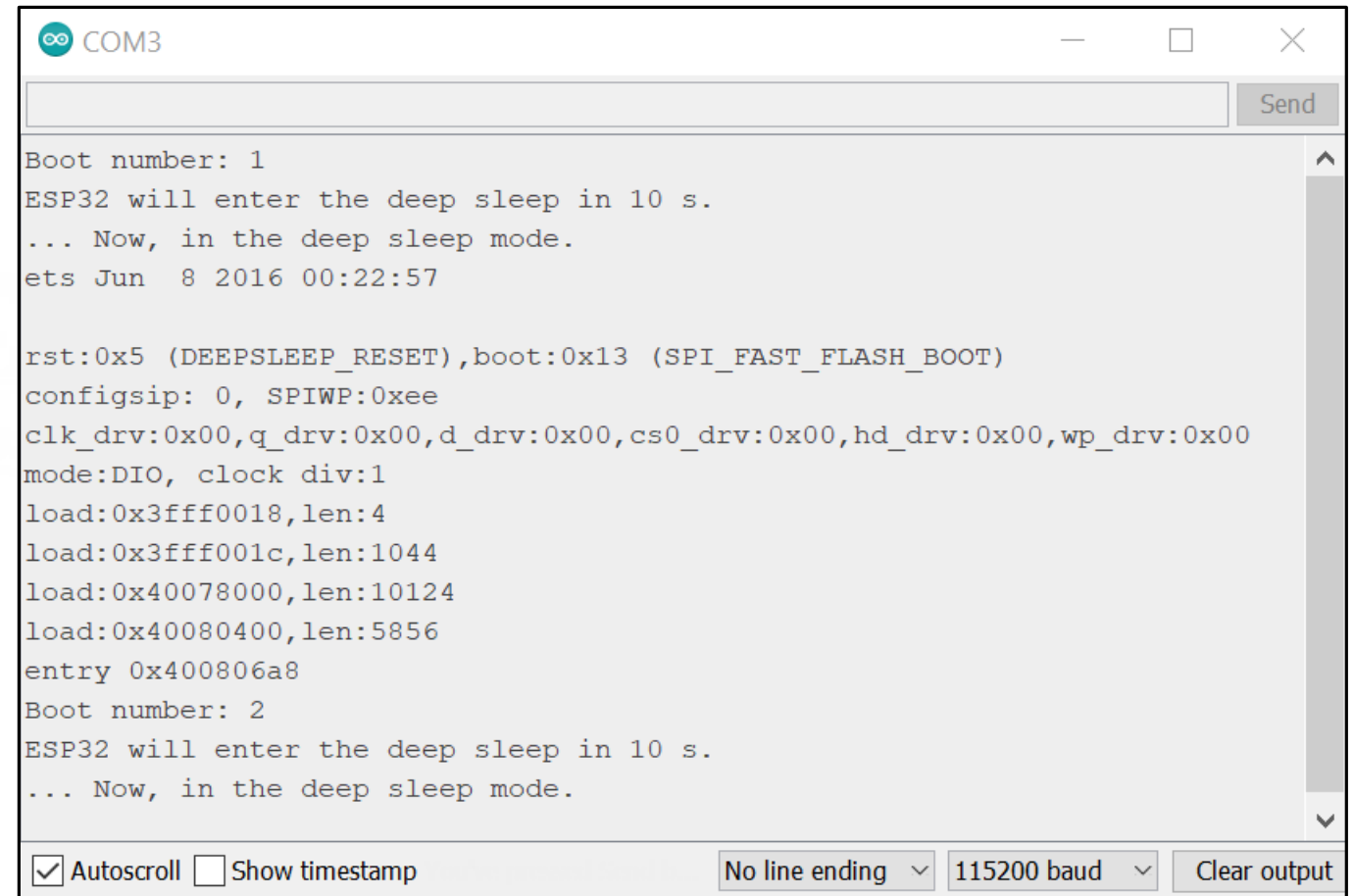
Using the deep sleep.

```
1 #define TIME_TO_SLEEP 5
2 RTC_DATA_ATTR int bootCount = 0;
3
4 void setup() {
5     // Declare the timer wake up Declare the timer wake up.
6     esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP*1000000);
7
8     // Tasks to do before going to deep sleep
9     Serial.begin(115200);
10    delay(1000); //Take some time to open up the Serial Monitor
11    ++bootCount;
12    Serial.println("Boot number: " + String(bootCount));
13    Serial.println("ESP32 will enter the deep sleep in 10 s.");
14    delay(10000);
15
16    //Go to sleep now
17    Serial.println("... Now, in the deep sleep mode.");
18    esp_deep_sleep_start();
19    Serial.println("This sentence will never be printed.");
20 }
21
22 void loop() {
23     //ESP32 will never come here.
24 }
```

3.3 Timer Wake-Up

Example 7 (Ex_TimerWakeUp.ino)

The result will be displayed in the serial monitor as shown below. No circuit connection is needed.



```
COM3
Boot number: 1
ESP32 will enter the deep sleep in 10 s.
... Now, in the deep sleep mode.
ets Jun 8 2016 00:22:57

rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0018,len:4
load:0x3fff001c,len:1044
load:0x40078000,len:10124
load:0x40080400,len:5856
entry 0x400806a8
Boot number: 2
ESP32 will enter the deep sleep in 10 s.
... Now, in the deep sleep mode.
```

☒ Autoscroll ☐ Show timestamp No line ending 115200 baud Clear output

3.4 Touch Wake-Up

1. The Commands

You can wake up the ESP32 from deep sleep using the touch pins (touch sensors). However, these pins must be set as interrupts. The following commands will be used for touch wake up.

- 1) We need to **set the touch pin that we will use to wake ESP32 as an interrupt** by the command with these 3 input parameters:

touchAttachInterrupt(Tpin, callback, Threshold);

where *Tpin* is the GPIO number of a touch pin, *callback* is the function name, and *Threshold* is a value. **If the value from *Tpin* is lower than *Threshold*, the interrupt happens and the callback function will be executed.**

- 2) Next, we need to use the this function:

esp_sleep_enable_touchpad_wakeup();

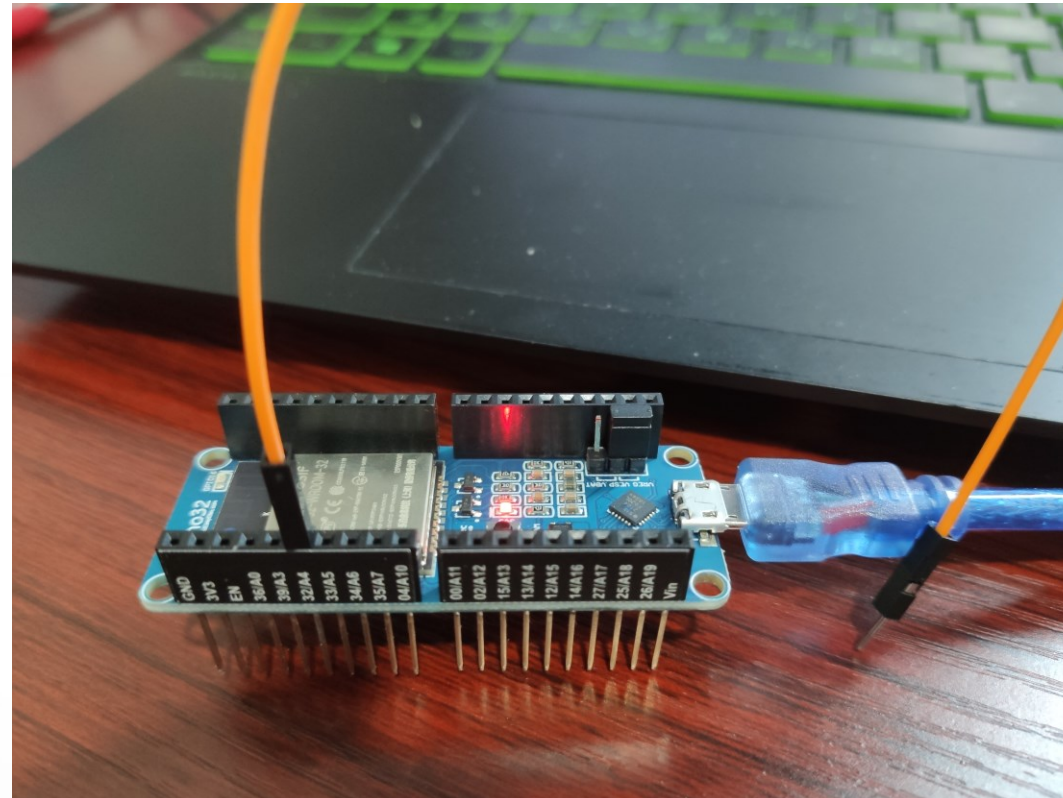
This function to set the touch pins as a wake up source.

Remark: **We do not need to declare the pin mode.**

3.4 Touch Wake-Up

Example 8 (Ex_TouchWakeUp.ino)

We will write an Arduino program such that ESP32 will go to the deep sleep and will be waken up when we touch the GPIO32 pin (a touch pin) by using the touch wake up. Here, we will connect a wire to the GPIO32 pin as shown below.



```
1 #define Threshold 40 // higher value, higher sensitivity
2 RTC_DATA_ATTR int bootCount = 0;
3
4 void callback(){
5     //placeholder callback function
6 }
7 void setup(){
8     // Declare the touch wake up - use GPIO32 (TOUCH9)
9     touchAttachInterrupt(32, callback, Threshold);
10    esp_sleep_enable_touchpad_wakeup();
11
12    // Tasks to do before going to deep sleep
13    Serial.begin(115200);
14    delay(1000); //Take some time to open up the Serial Monitor
15    ++bootCount;
16    Serial.println("Boot number: " + String(bootCount));
17    Serial.println("ESP32 will enter the deep sleep in 10 s.");
18    delay(10000);
19
20    //Go to sleep now
21    Serial.println("... Now, in the deep sleep mode.");
22    esp_deep_sleep_start();
23    Serial.println("This sentence will never be printed.");
24 }
25 void loop(){
26     //ESP32 will never come here.
27 }
```

Two commands together.

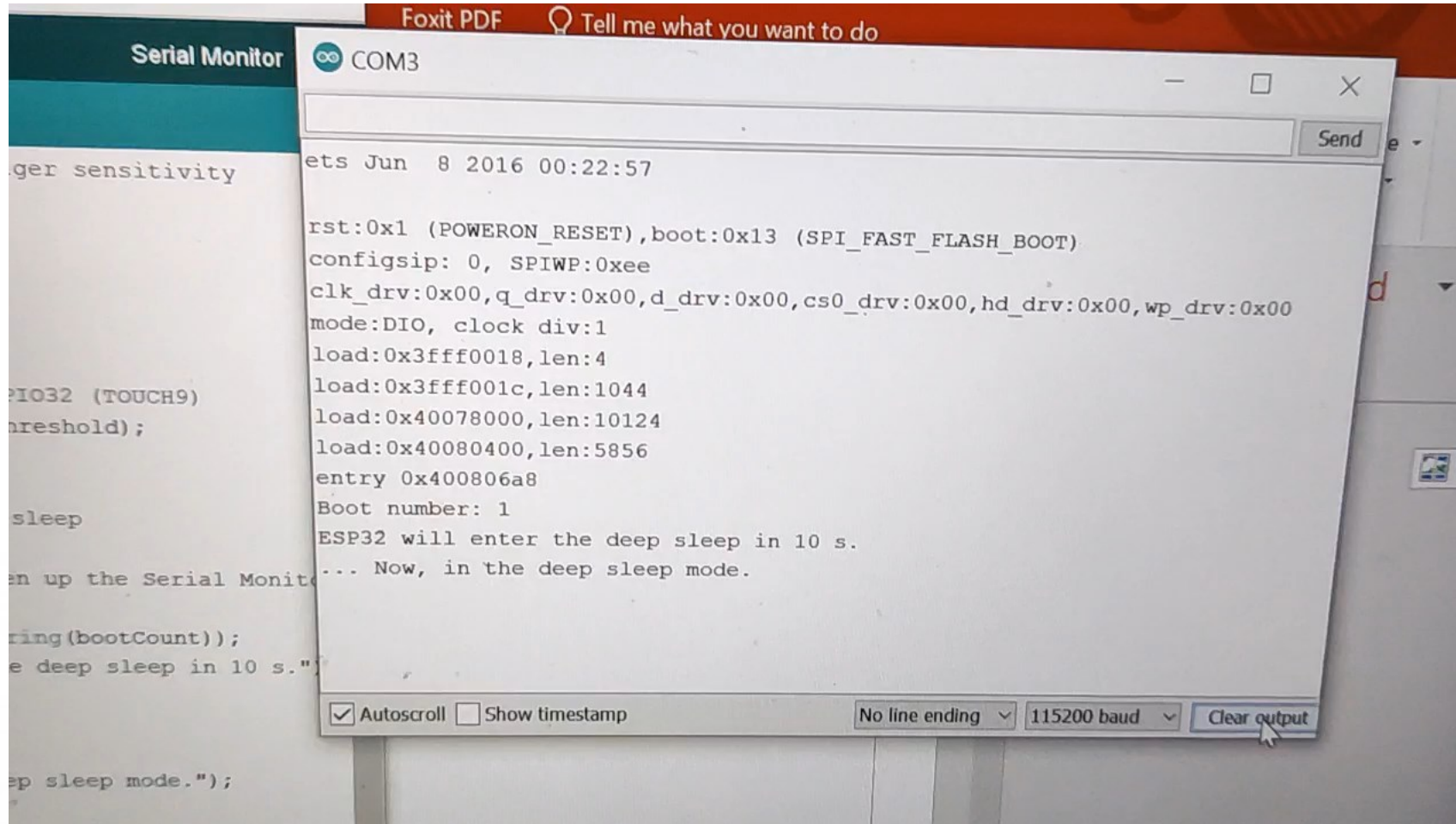
Before

Using the deep sleep.

3.4 Touch Wake-Up

Example 8 (Ex_TouchWakeUp.ino)

A demonstration video is shown below.



```
ets Jun 8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
config: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0018,len:4
load:0x3fff001c,len:1044
load:0x40078000,len:10124
load:0x40080400,len:5856
entry 0x400806a8
Boot number: 1
ESP32 will enter the deep sleep in 10 s.
... Now, in the deep sleep mode.
```

3.5 ext0 External Wake-Up

1. The Command

- The following GPIO pins can be used as ext0 external wake up: GPIOs 0, 2, 4, 12-15, 25-27, 32-39. Note that we do not need to declare the pin mode.
- We use the following command to enable the ext0 external wake-up source and put it before calling the *esp_deep_sleep_start()* function:

esp_sleep_enable_ext0_wakeup(GPIO_NUM_X, LOGIC_LEVEL);

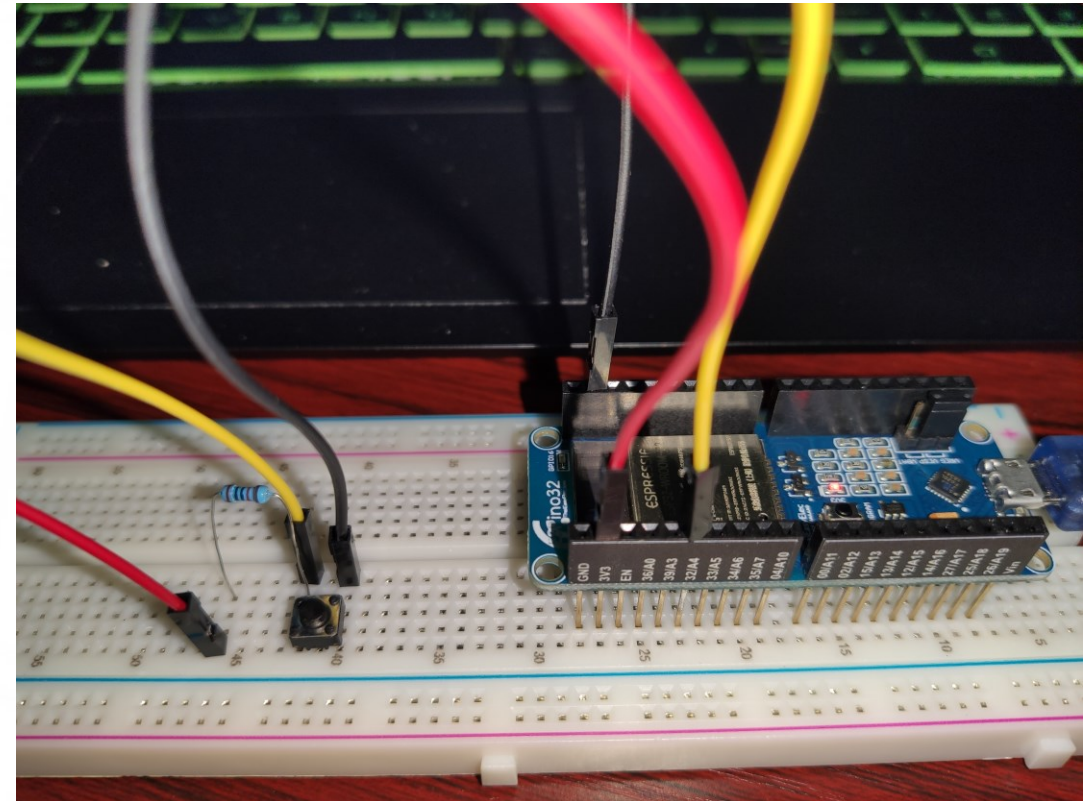
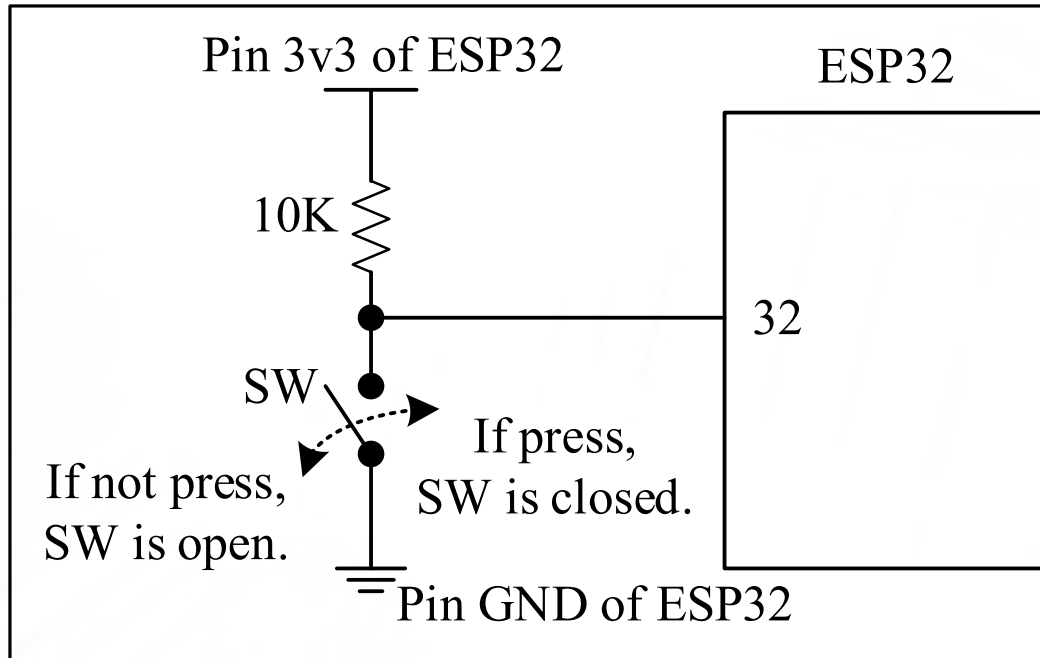
The function takes two parameters.

- ✓ *GPIO_NUM_X*: *X* is the GPIO number which we will use to wake ESP32.
- ✓ *LOGIC_LEVEL*: decides if we want to trigger the wake up by a **LOW** or a **HIGH** state of the pin.
- Example: This Arduino command:
esp_sleep_enable_ext0_wakeup(GPIO_NUM_32, LOW);
means that “when the voltage across the pin 32 is LOW, the ESP32 will wake up.”

3.5 ext0 External Wake-Up

Example 9 (Ex_ext0WakeUp.ino)

We connect a switch to the GPIO32 as shown below and write an Arduino program to wake ESP32 from deep sleep by pressing the switch.



3.5 ext0 External Wake-Up

Example 9 (Ex_ext0WakeUp.ino)

```
1 RTC_DATA_ATTR int bootCount = 0;
2
3 void setup() {
4   // Declare the ext0 wake up - use GPIO32
5   esp_sleep_enable_ext0_wakeup(GPIO_NUM_32, LOW);
6
7   // Tasks to do before going to deep sleep
8   Serial.begin(115200);
9   delay(1000); //Take some time to open up the Serial Monitor
10  ++bootCount;
11  Serial.println("Boot number: " + String(bootCount));
12  Serial.println("ESP32 will enter the deep sleep in 10 s.");
13  delay(10000);
14
15  //Go to sleep now
16  Serial.println("... Now, in the deep sleep mode.");
17  esp_deep_sleep_start();
18  Serial.println("This sentence will never be printed.");
19 }
20
21 void loop() {
22   //ESP32 will never come here.
23 }
```

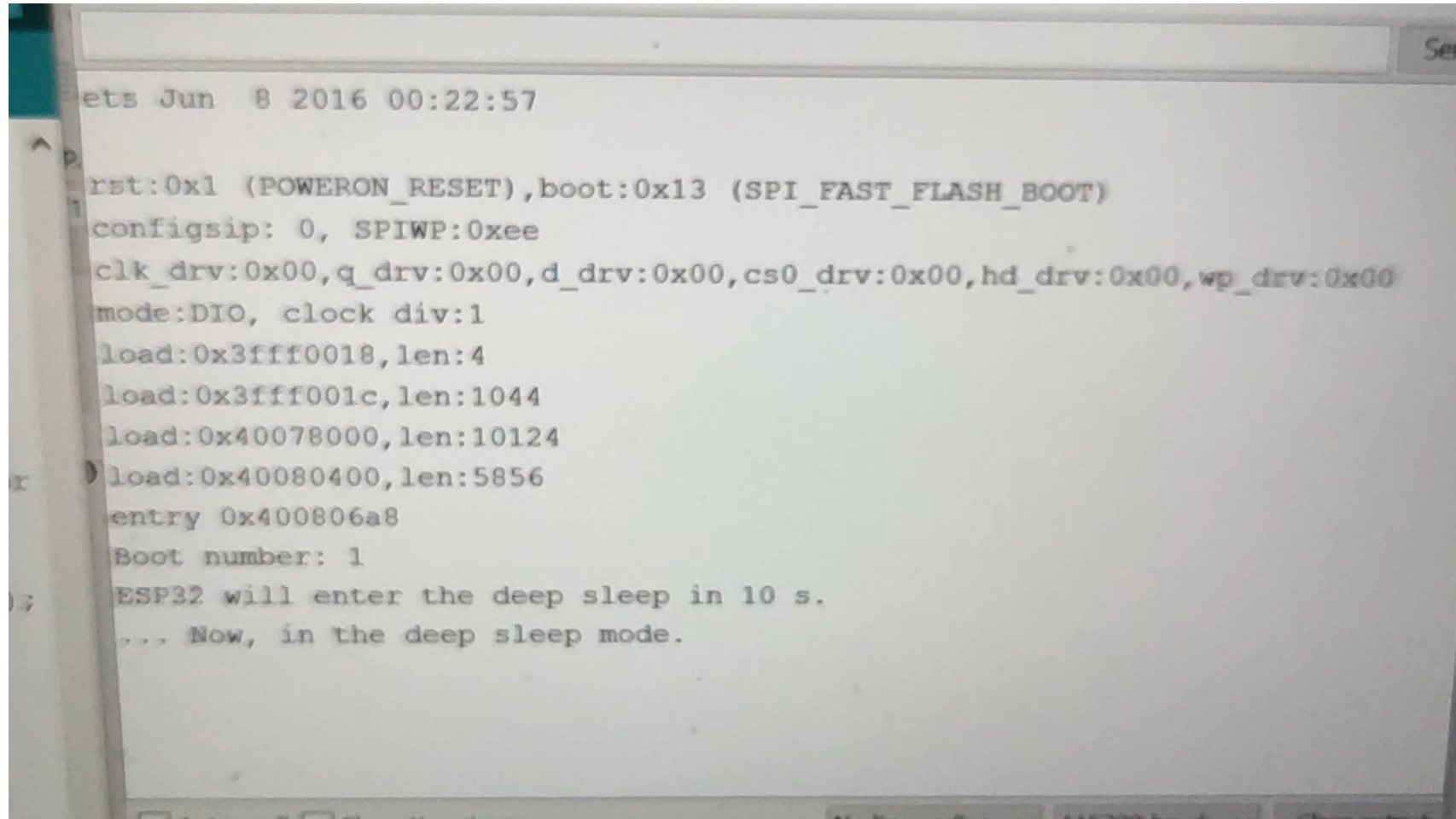
Using the Ext0 wake-up.

Before

Using the deep sleep.

3.5 ext0 External Wake-Up Example 9 (Ex_ext0WakeUp.ino)

A demonstration video is shown below.



```
ets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0018,len:4
load:0x3fff001c,len:1044
load:0x40078000,len:10124
load:0x40080400,len:5856
entry 0x400806a8
Boot number: 1
ESP32 will enter the deep sleep in 10 s.
... Now, in the deep sleep mode.
```

3.6 ext1 External Wake-Up

1. The Command

- The ext1 external wake up is for the cases that we would like to have many pins to wake ESP32 up. There are two options:
 - 1) One of these pins can wake ESP32 up.
 - 2) All of these together (= AND logic) will wake ESP32 up.

3.6 ext1 External Wake-Up

1. The Commands

- The following GPIO pins can be used as ext1 external wake up: GPIOs 0, 2, 4, 12-15, 25-27, 32-39. Note that we do not need to declare the pin mode.
- We use the following command to enable the ext1 external wake-up source and put it before calling the *esp_deep_sleep_start()* function:

esp_sleep_enable_ext1_wakeup(BUTTON_PIN_MASK, LOGIC_LEVEL);

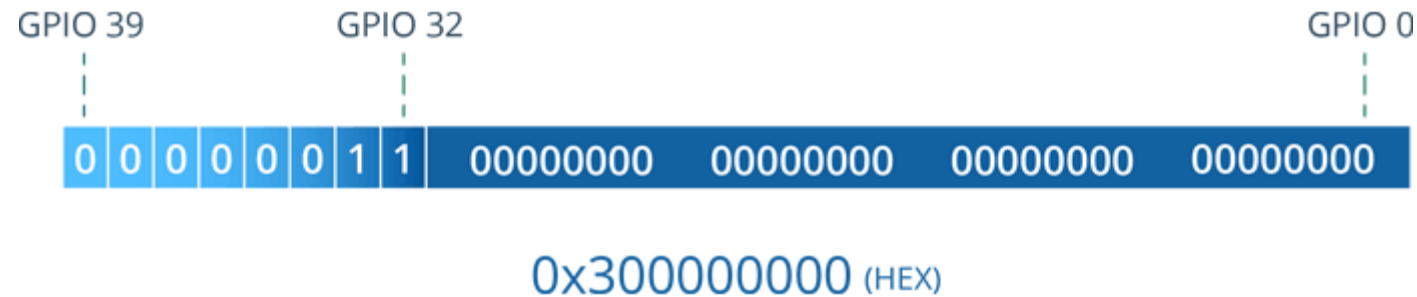
The function takes two parameters.

- ✓ *BUTTON_PIN_MASK*: we use to specify which pins will be used.
- ✓ *LOGIC_LEVEL*: specifies which option that we will use to wake ESP32 up.

3.6 ext1 External Wake-Up

1. The Commands

- The *BUTTON_PIN_MASK* parameter is to identify which pins will be used. It can be written as a Hex number. The GPIO pins can be mapped to 40 bits as shown below.



- The MSB bit is for GPIO39. The LSB bit is for GPIO0.
- Bit masked with 1 to specify its usage in the ext1 external wake up.
- Above, if we want to use GPIO32 and GPIO33, the *BUTTON_PIN_MASK* will be set to 0x0300000000.

3.6 ext1 External Wake-Up

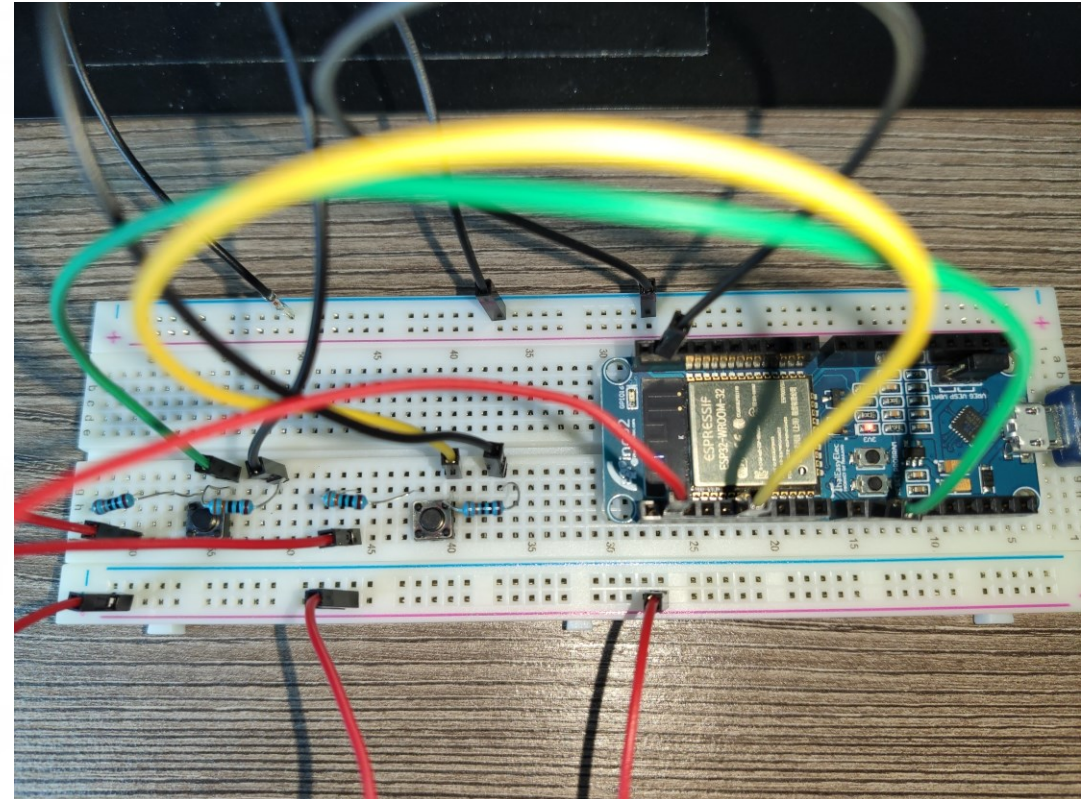
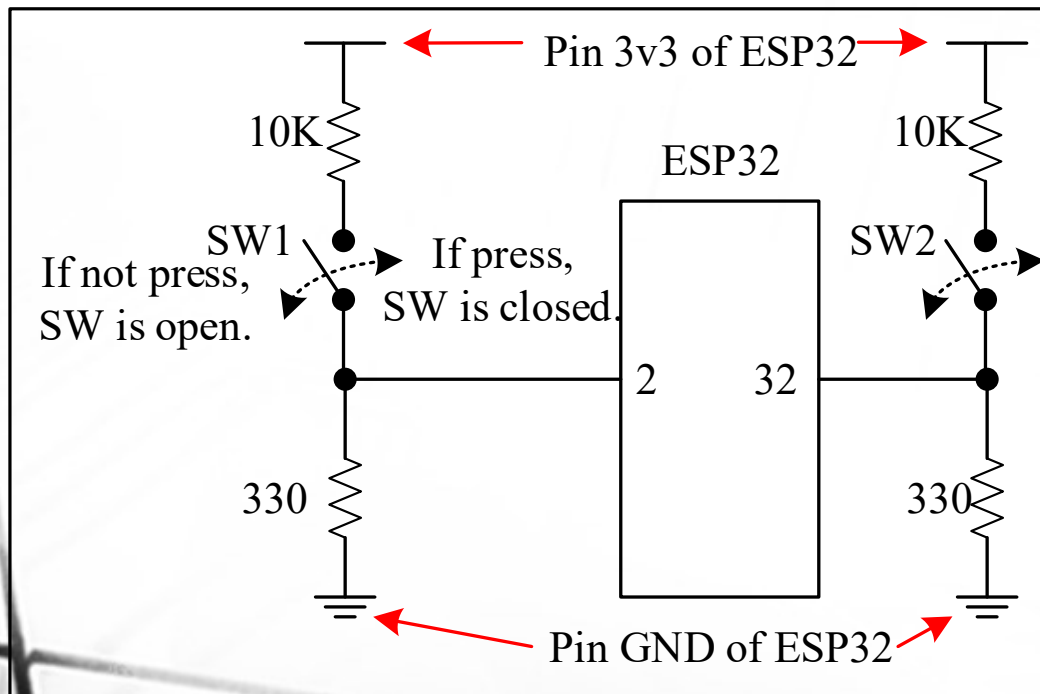
1. The Commands

- The *LOGIC_LEVEL* parameter specifies which option that we will use to wake ESP32 up.
 - If the *LOGIC_LEVEL* parameter is *ESP_EXT1_WAKEUP_ANY_HIGH*, we will wake ESP32 up if **one** of the selected pins is **high**.
 - ✓ Example: If the *BUTTON_PIN_MASK* is set to 0x0300000000, the high voltage at either pin 32 or 33 will wake ESP32.
 - If the *LOGIC_LEVEL* parameter is *ESP_EXT1_WAKEUP_ALL_LOW*, we will wake ESP32 up if **all** the selected pins are **low**.
 - ✓ Example: If the *BUTTON_PIN_MASK* is set to 0x0300000000, the low voltages at both pin 32 and 33 will wake ESP32.

3.6 ext1 External Wake-Up

Example 10 (Ex_ext1WakeUp.ino)

We connect two switches to the GPIO02 and GPIO32 as shown below and write an Arduino program to wake ESP32 from deep sleep by pressing one of these switches. Note that, by connecting the switches in this way, pressing the switch gives the *HIGH* logic at the ESP32 pin. Since we will use GPIO2 and GPIO32 for ext1 wake up, the *BUTTON_PIN_MASK* parameter will be set to 0x0100000004.



3.6 ext1 External Wake-Up

Example 10 (Ex_ext1WakeUp.ino)

```
1 RTC_DATA_ATTR int bootCount = 0;
2
3 void setup() {
4   // Declare the ext1 wake up - use GPIO2 and GPIO32
5   esp_sleep_enable_ext1_wakeup(0x0100000004, ESP_EXT1_WAKEUP_ANY_HIGH);
6
7   // Tasks to do before going to deep sleep
8   Serial.begin(115200);
9   delay(1000); //Take some time to open up the Serial Monitor
10  ++bootCount;
11  Serial.println("Boot number: " + String(bootCount));
12  Serial.println("ESP32 will enter the deep sleep in 10 s.");
13  delay(10000);
14
15  //Go to sleep now
16  Serial.println("... Now, in the deep sleep mode.");
17  esp_deep_sleep_start();
18  Serial.println("This sentence will never be printed.");
19 }
20
21 void loop() {
22   //ESP32 will never come here.
23 }
```

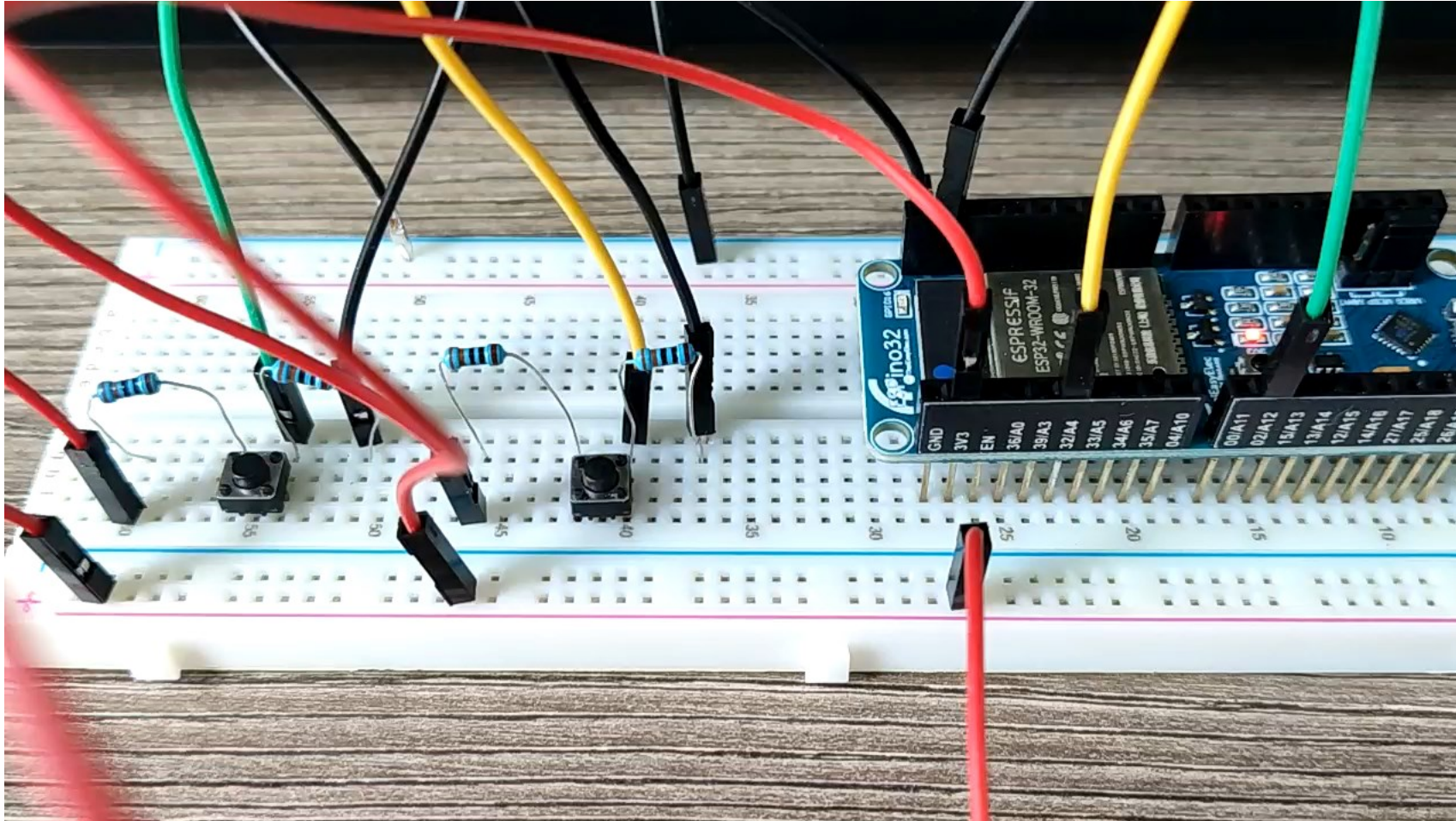
Using the Ext1 wake-up.

Before

Using the deep sleep.

3.6 ext1 External Wake-Up Example 10 (Ex_ext1WakeUp.ino)

A demonstration video is shown below.



Topic	Detail	Command
3.2 Deep Sleep	Go to deep sleep	<code>esp_deep_sleep_start();</code>
3.3 Timer Wake-Up	Wake ESP32 by using a timer	<code>esp_sleep_enable_timer_wakeup(X*1000000) ;</code>
3.4 Touch Wake-Up	Wake ESP32 by using a Touch pin	<code>touchAttachInterrupt(Tpin, callback, Threshold);</code>
		<code>esp_sleep_enable_touchpad_wakeup();</code>
3.5 ext0 External Wake-Up	Wake ESP32 by using a pin	<code>esp_sleep_enable_ext0_wakeup(GPIO_NUM_X, LOGIC_LEVEL);</code>
3.6 ext1 External Wake-Up	Wake ESP32 by using a set of pins	<code>esp_sleep_enable_ext1_wakeup(BUTTON_PIN_MASK, LOGIC_LEVEL);</code>

4. Advanced Topics

4. Advanced Topics

- 1) ESP32 Flash Memory – Store Permanent Data (Write and Read):
<https://randomnerdtutorials.com/esp32-flash-memory/>
- 2) ESP32 Over-the-air (OTA) Programming – Web Updater Arduino IDE:
<https://randomnerdtutorials.com/esp32-over-the-air-ota-programming/>
- 3) Get Epoch/Unix Time with the ESP32:
<https://randomnerdtutorials.com/epoch-unix-time-esp32-arduino/>
- 4) Etc.

Questions?